

CSE4067

Introduction to Blockchain Programming

Goshgar Can Ismayilov

goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

15.10.2024

Preimage Resistant

Definition:

- h is a hash function
- for essentially all pre-specified outputs y , it is computationally infeasible to find an x such that $h(x) = y$
- h is also called a **one-way function**.

One-Way Function



output space

If a function h is a one-way-function, then a function h^{-1} does not exist. It is therefore computationally infeasible to find the input to an output of h .

Hash function⁻¹

?

Computationally infeasible?

Infeasible computation is a “hard” instance of an NP-Hard problem of sufficient size. “Hard” means that there is no better way than trying all possible solutions. Sufficient means, that the size is large enough that it can be considered not computable with classical computers, e.g. size = 256 $\rightarrow 2^{256}$.

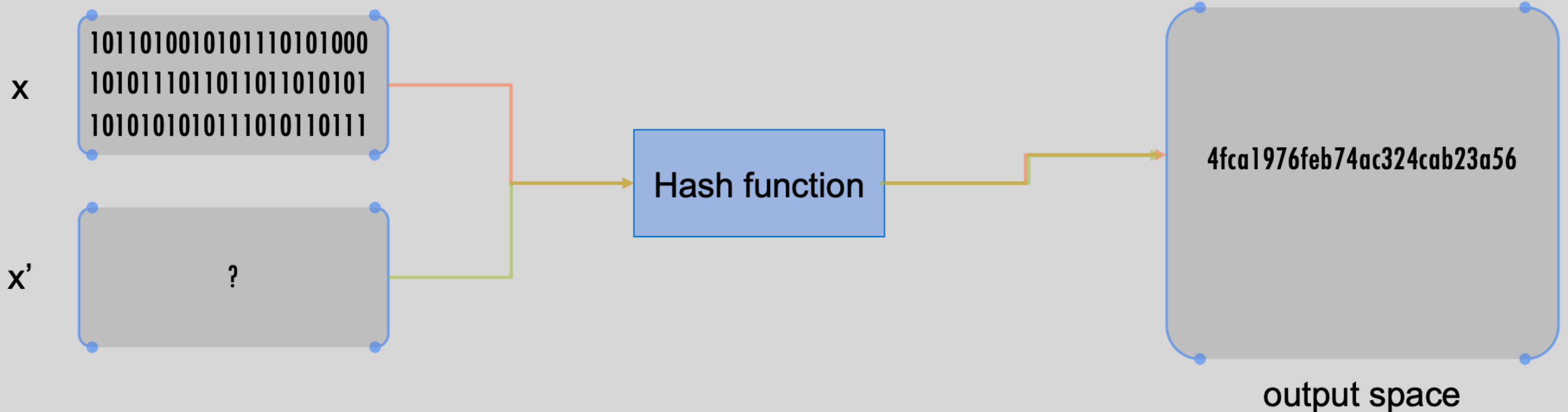
2nd Preimage Resistant

Definition:

- Given x it is computationally infeasible to **find any second input x'** with $x \neq x'$ such that $h(x) = h(x')$.

2nd Pre-image Resistance

It is computationally infeasible to find an x' which computes to the same hash output.

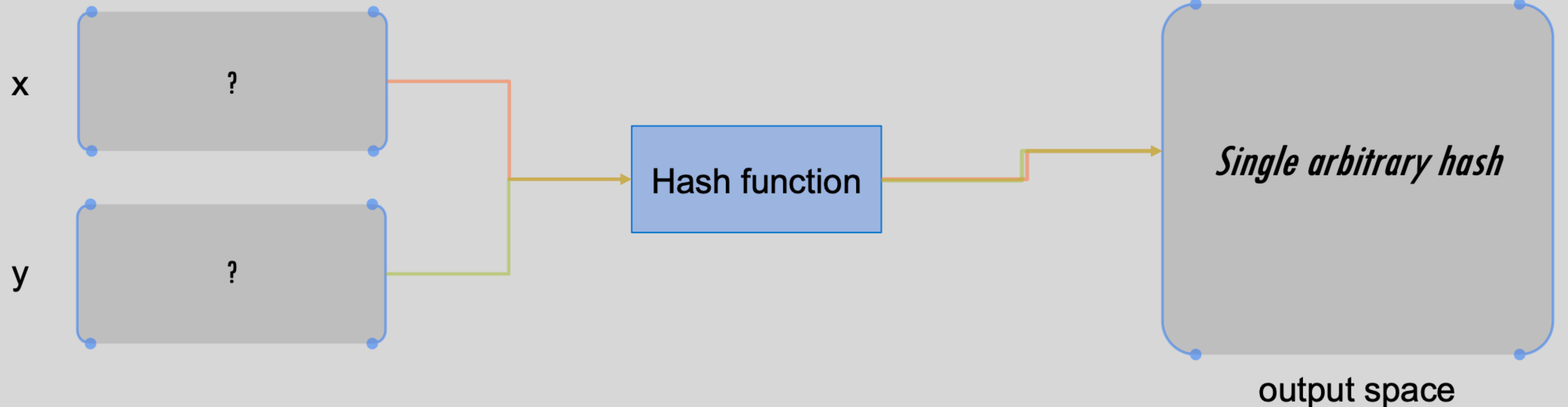


Collision Resistant

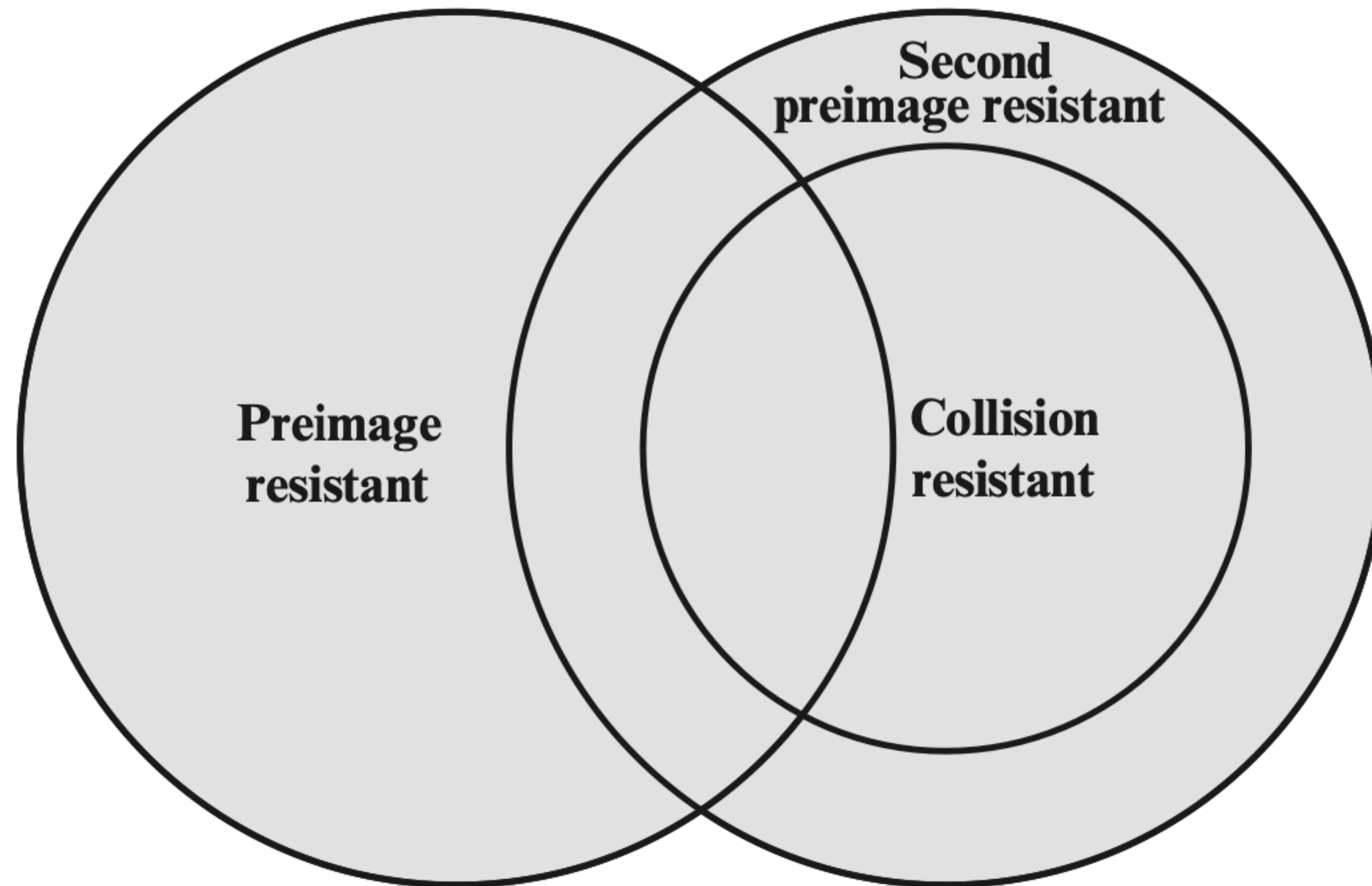
Definition:

- A hash function h is said to be **collision resistant** if it is infeasible to find two values, x and y , such that $x \neq y$, yet $h(x) = h(y)$.

Collision Resistance



Relations between Requirements



A Simple Hash Function

One of the simplest hash functions is the bit-by-bit exclusive-OR (XOR) of every block. This can be expressed as

$$C_i = b_{i1} \oplus b_{i2} \oplus \cdots \oplus b_{im}$$

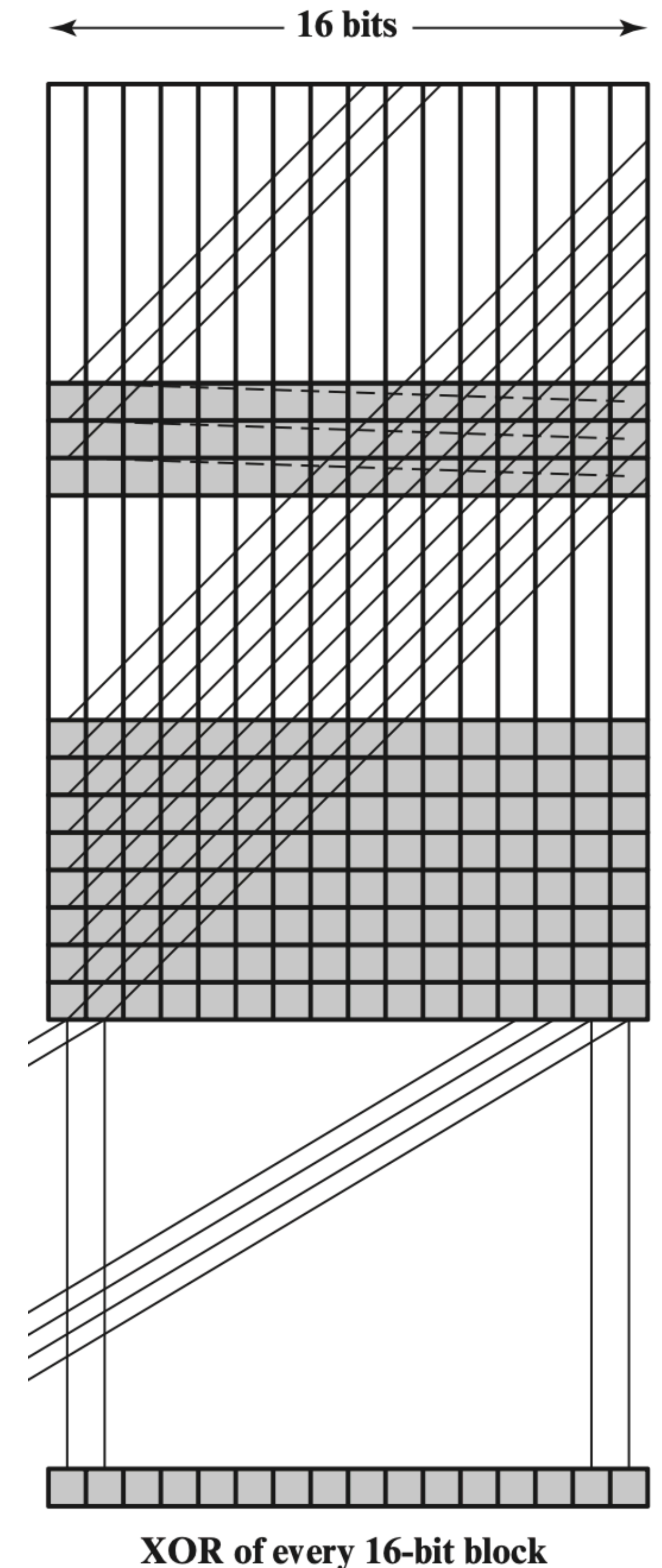
where

C_i = i th bit of the hash code, $1 \leq i \leq n$

m = number of n -bit blocks in the input

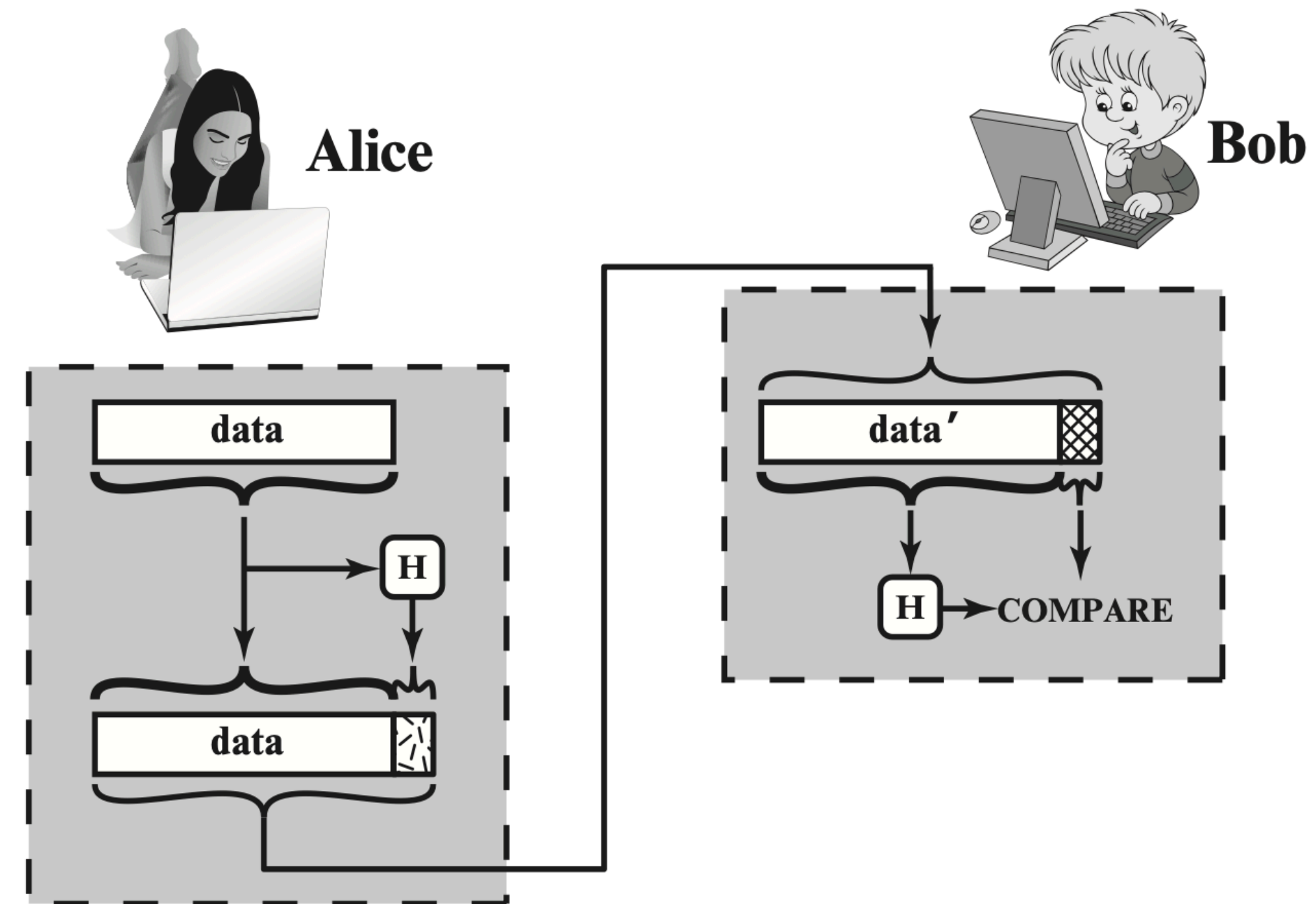
b_{ij} = i th bit in j th block

$\oplus$ = XOR operation

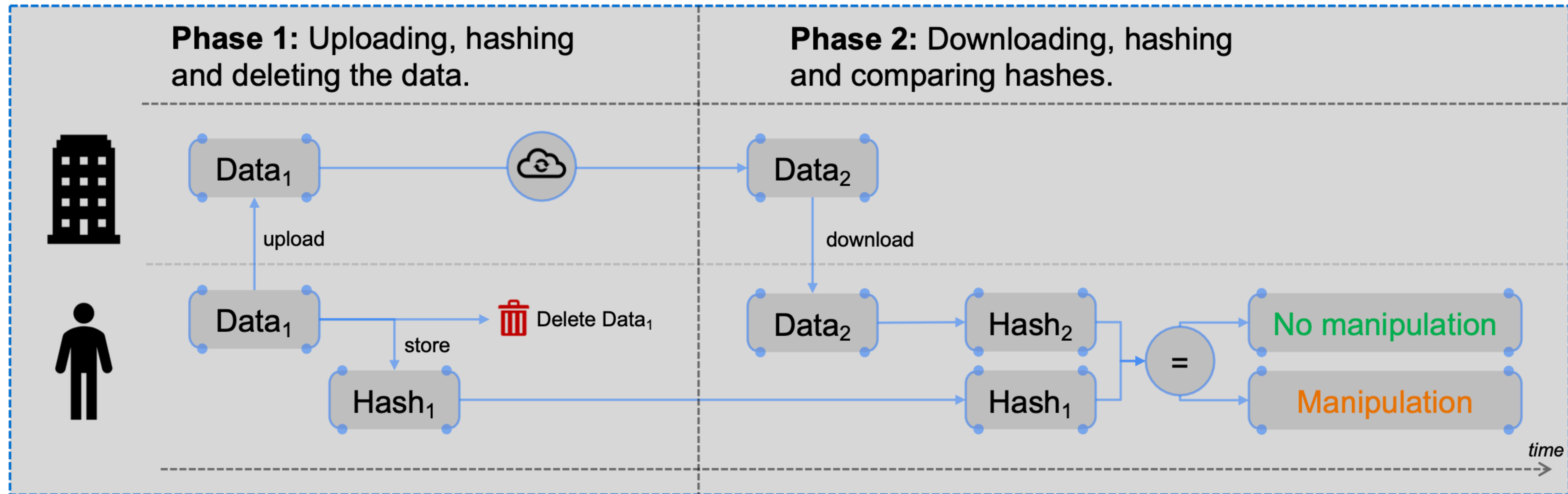


Hash Function: Message Authentication

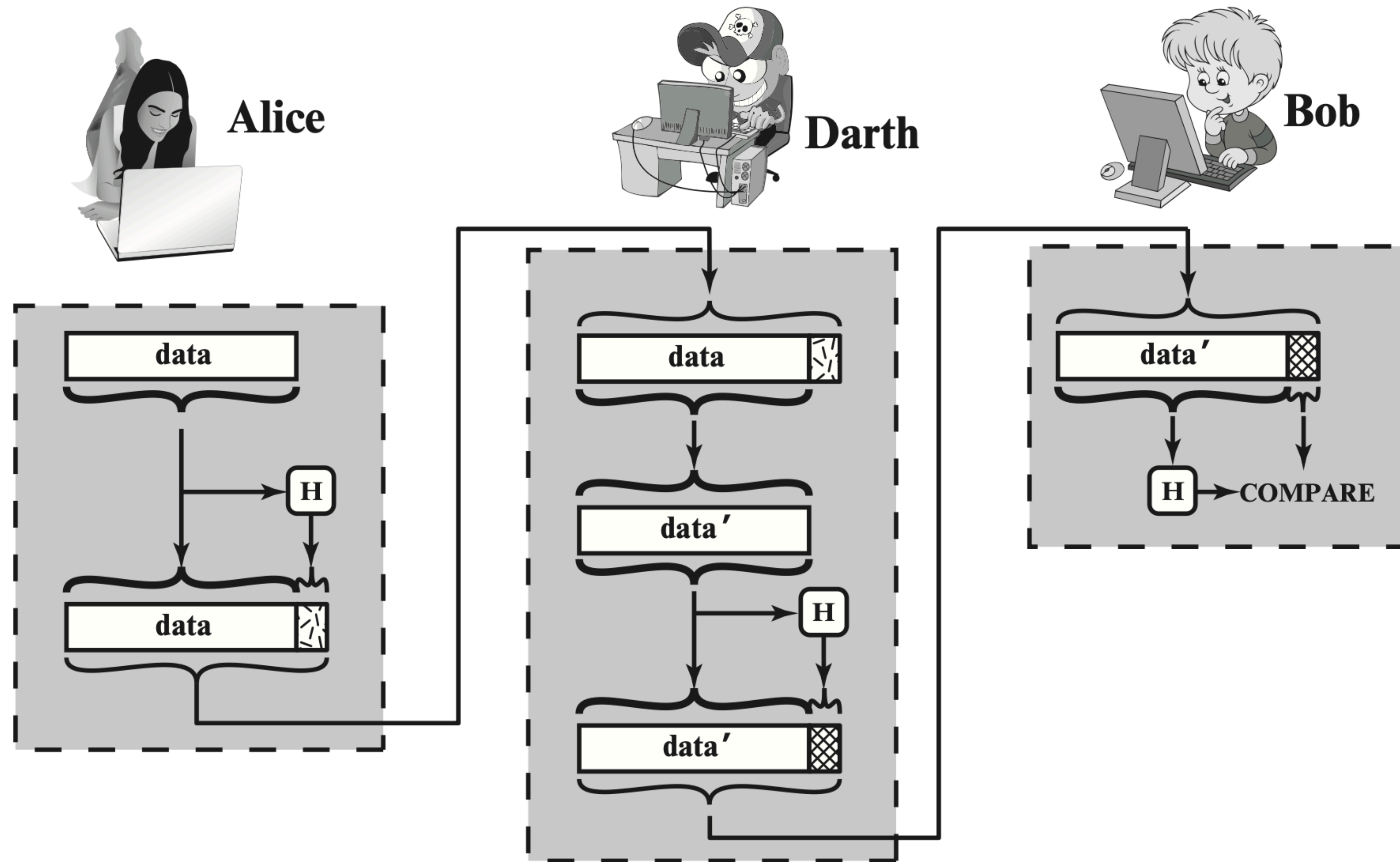
- Message authentication is a mechanism or service used to verify the integrity of a message.
- Message authentication assures that data received are exactly as sent (i.e., there is no modification, insertion, deletion, or replay).
- The hash function value is often referred to as a **message digest**.¹



Hash Function: Message Authentication

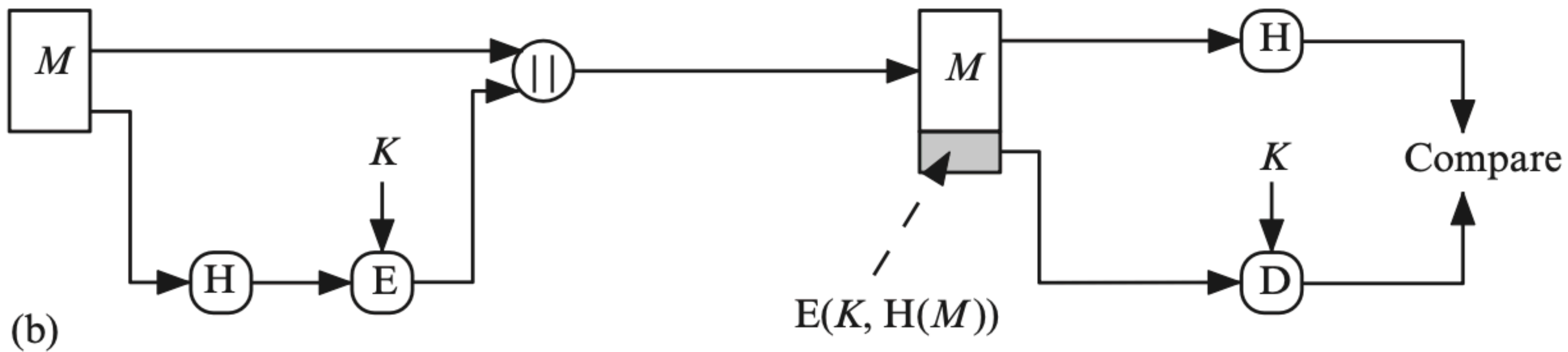
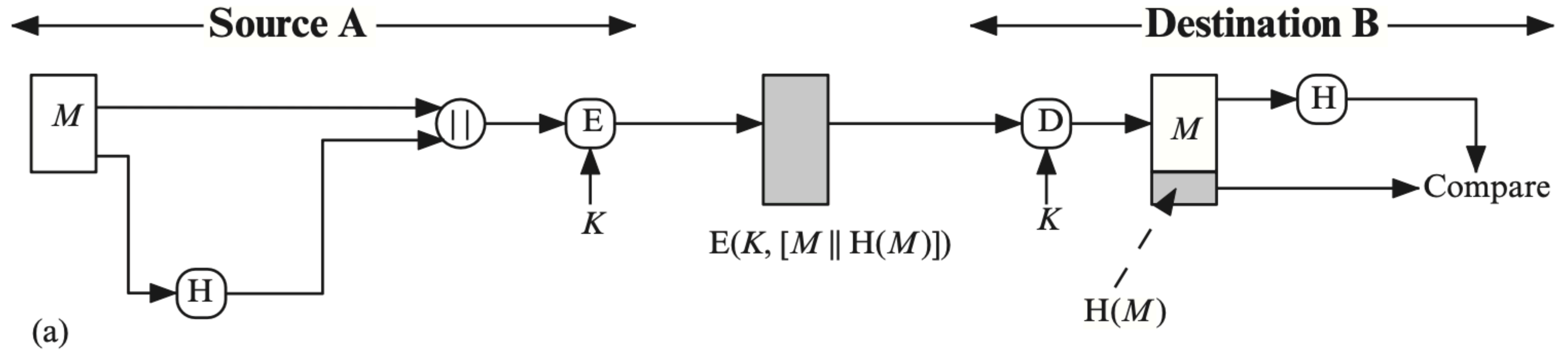


Hash Function: Message Authentication

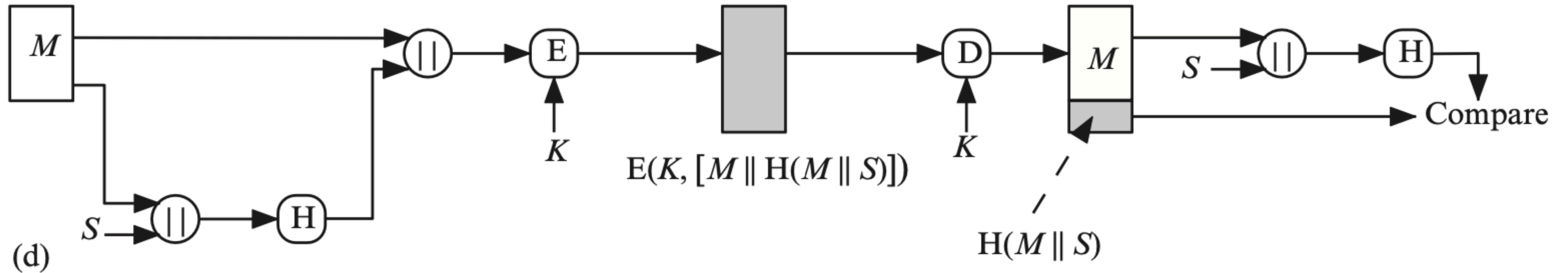
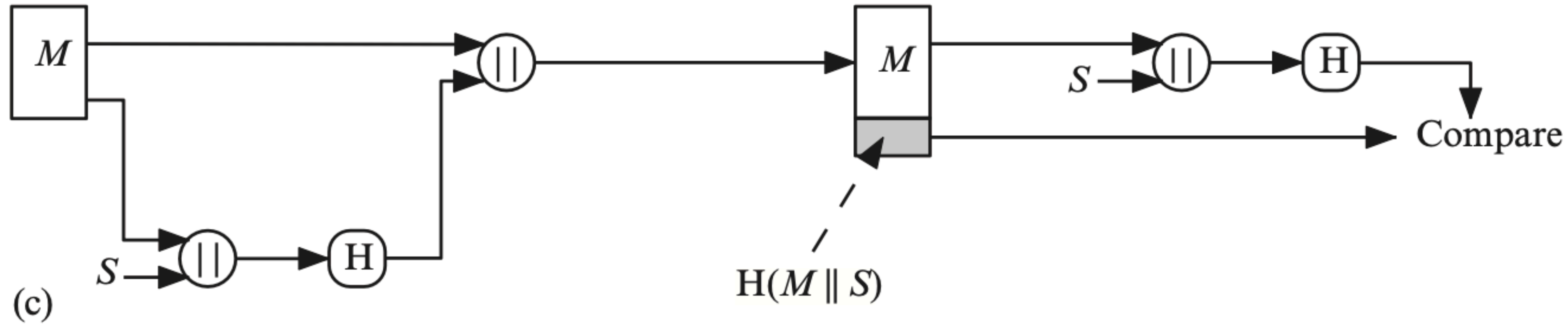


(b) Man-in-the-middle attack

Hash Function: Message Authentication

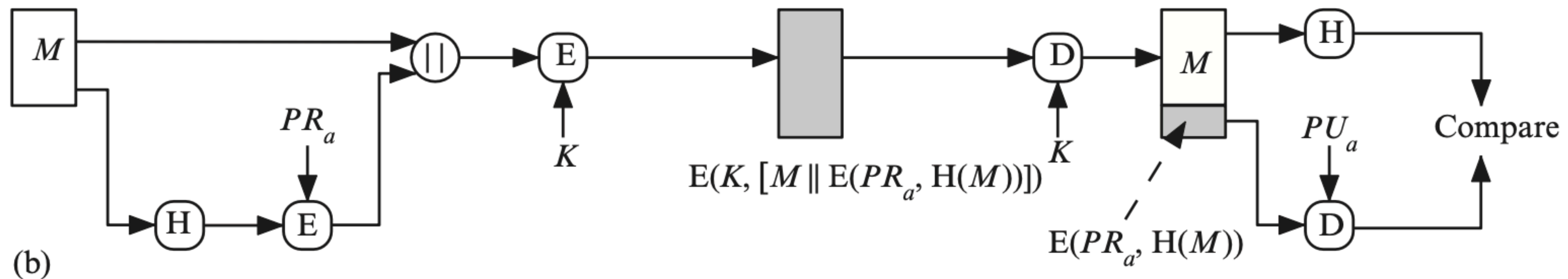
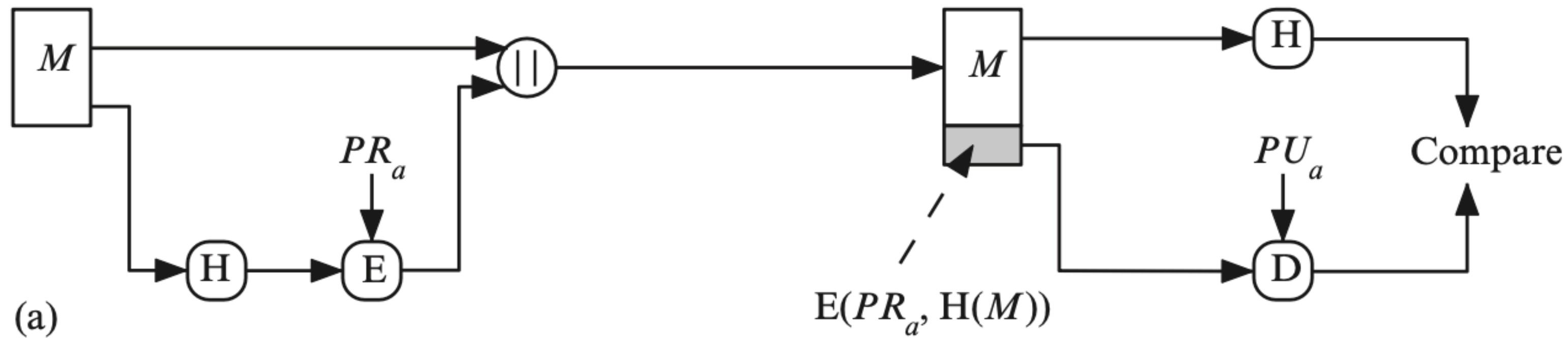


Hash Function: Message Authentication



Hash Function: Digital Signatures

← Source A → ← Destination B →

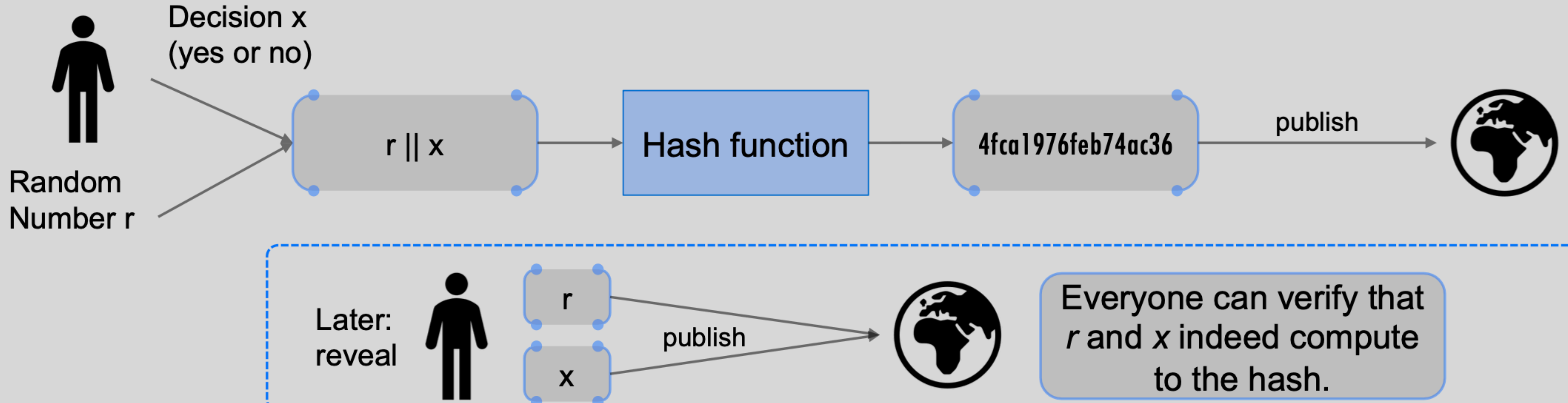


Hash Function: Commitment

A person can commit him/herself to a value without revealing it immediately.

Two algorithms of *Commitment Scheme*:

- $\text{com} := \text{commit}(\text{msg}, \text{nonce}^1)$
 - msg is the message and nonce is the random number. The hash of the concatenation is returned.
- $\text{verification} := \text{verify}(\text{com}, \text{msg}, \text{nonce})$
 - Checks and returns whether msg and nonce produce the same result as com .



Hash Function: Search Puzzle

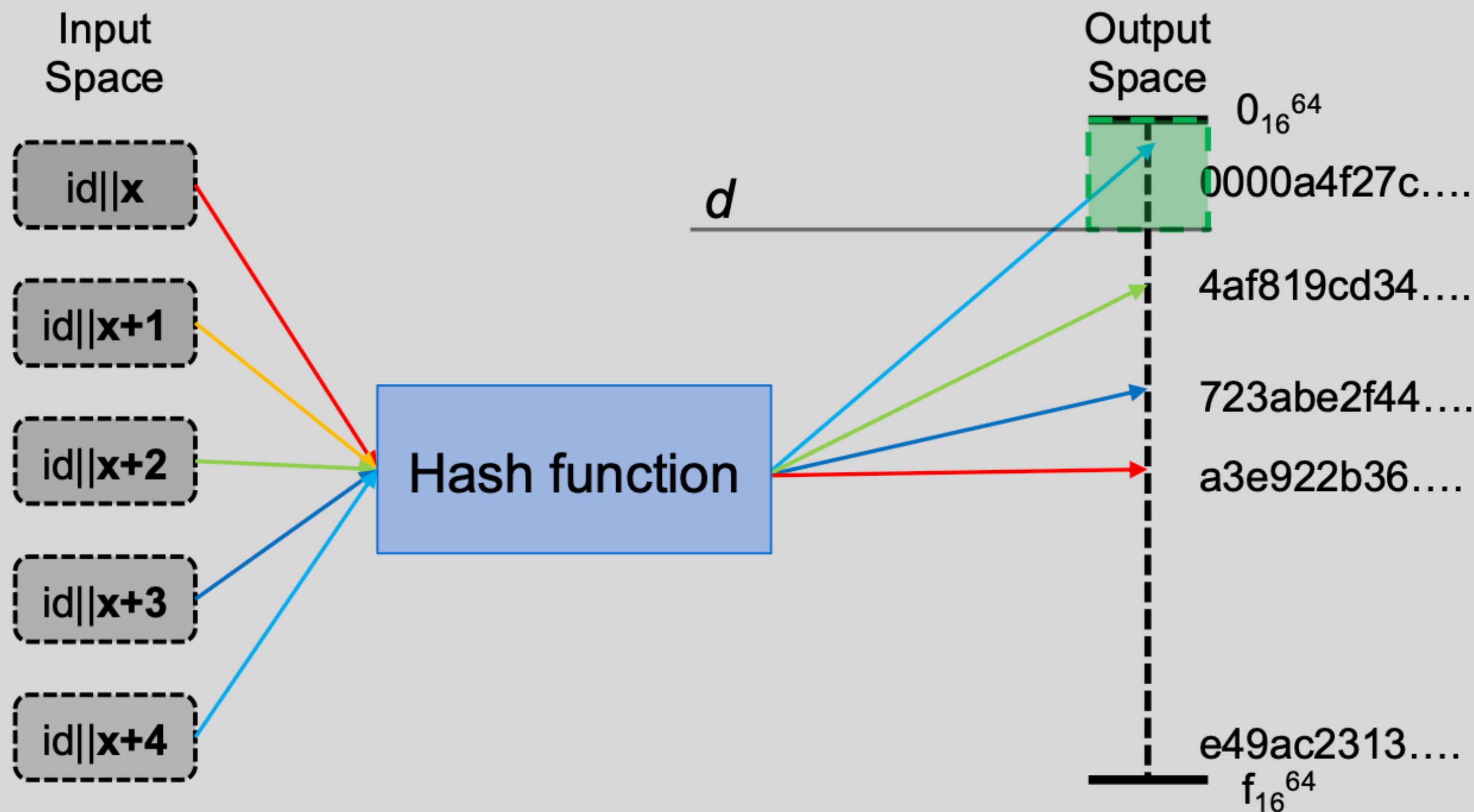
Search puzzle consists out of:

- **A hash function h**
 - Computes the *puzzle results*
- **A value id**
 - Is the *puzzle-ID* (makes puzzle solutions unique)
- **A target set Y**
 - For a valid solution, the *puzzle result* must lie within the target set Y
- **Computation**
 - The puzzle-ID is concatenated with a value x and hashed. x changes until the puzzle result lies within Y

- A search puzzle is a **mathematical problem** which **requires searching a very large space** in order to find a solution. In particular, there are **no shortcuts** in finding the solution.
- Hash function h produces an n -bit output. Therefore, it can take any of the 2^n input values.
- Solving the puzzle requires **finding an input so that the output falls within the set Y** . Depending on the size of the set Y , the puzzle can be more or less difficult, e.g., if the set contains all n -bit strings it is trivial, if it contains only one string, it is maximally hard.

Hash Function: Search Puzzle

For simplicity, we define the target set Y as $\{0, 1, \dots, d\}$, therefore we only have to check if the result of the hash function is smaller than the target difficulty d . We define the *puzzleID* as “BBSE”¹. A pseudocode that implements this search puzzle would look like the following.



```
puzzleID = "BBSE";
d = '0000f000000000000000...';
x = 0; //counter
while(true) {
    puzzleResult = hash(puzzleID||x);
    //if solution found, return
    if(puzzleResult < d) {return x;}
    x++;
}
```

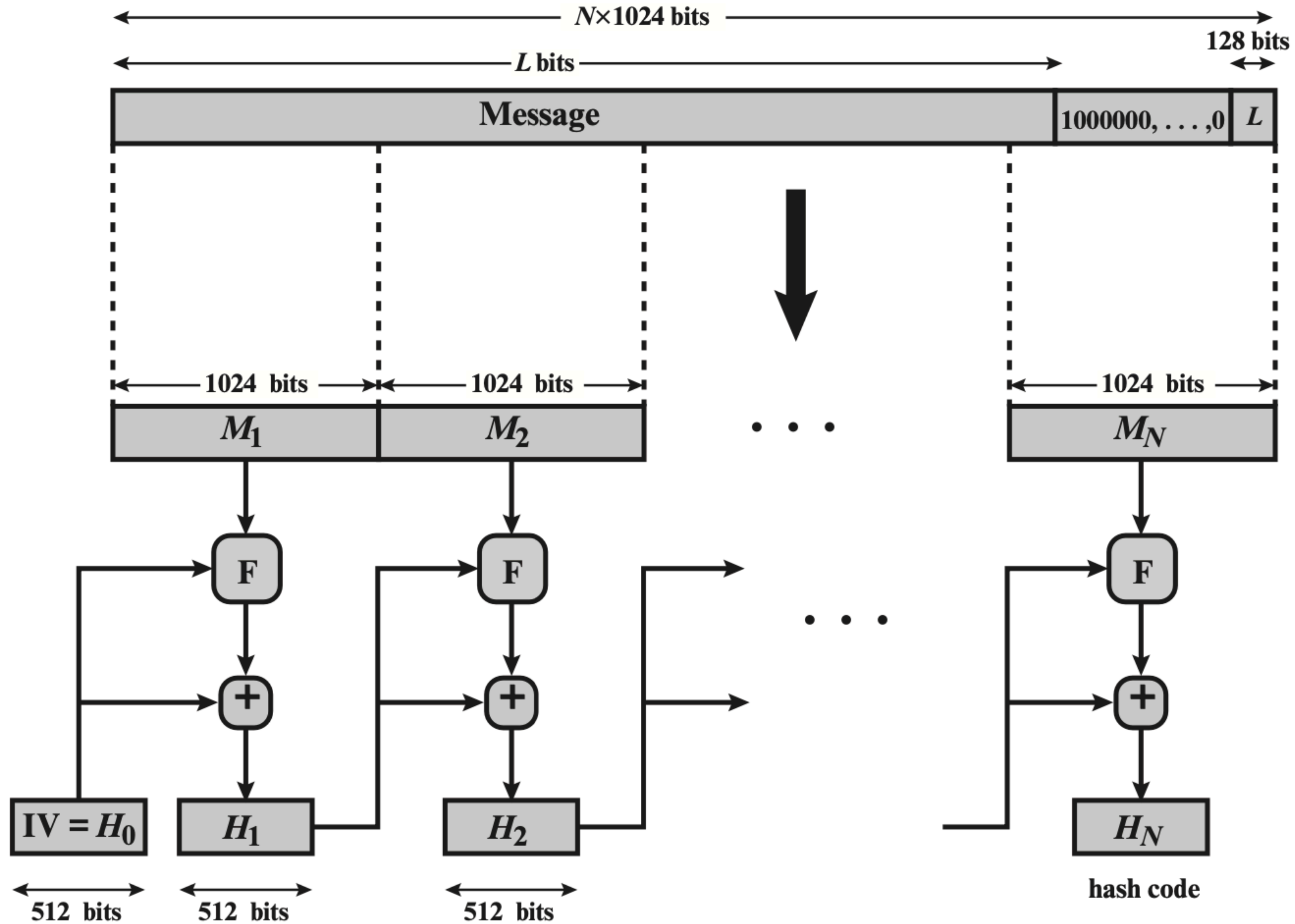
Current Secure Hash Functions

- Message Digest 4 / 5 (MD4 / MD5) **Considered broken!**
- Secure Hash Algorithm 1 (SHA-1) **Considered broken!**
- Secure Hash Algorithm 2 / 3 (SHA-2 / SHA-3) **At the moment safe to use, favor SHA-3 over SHA-2.**

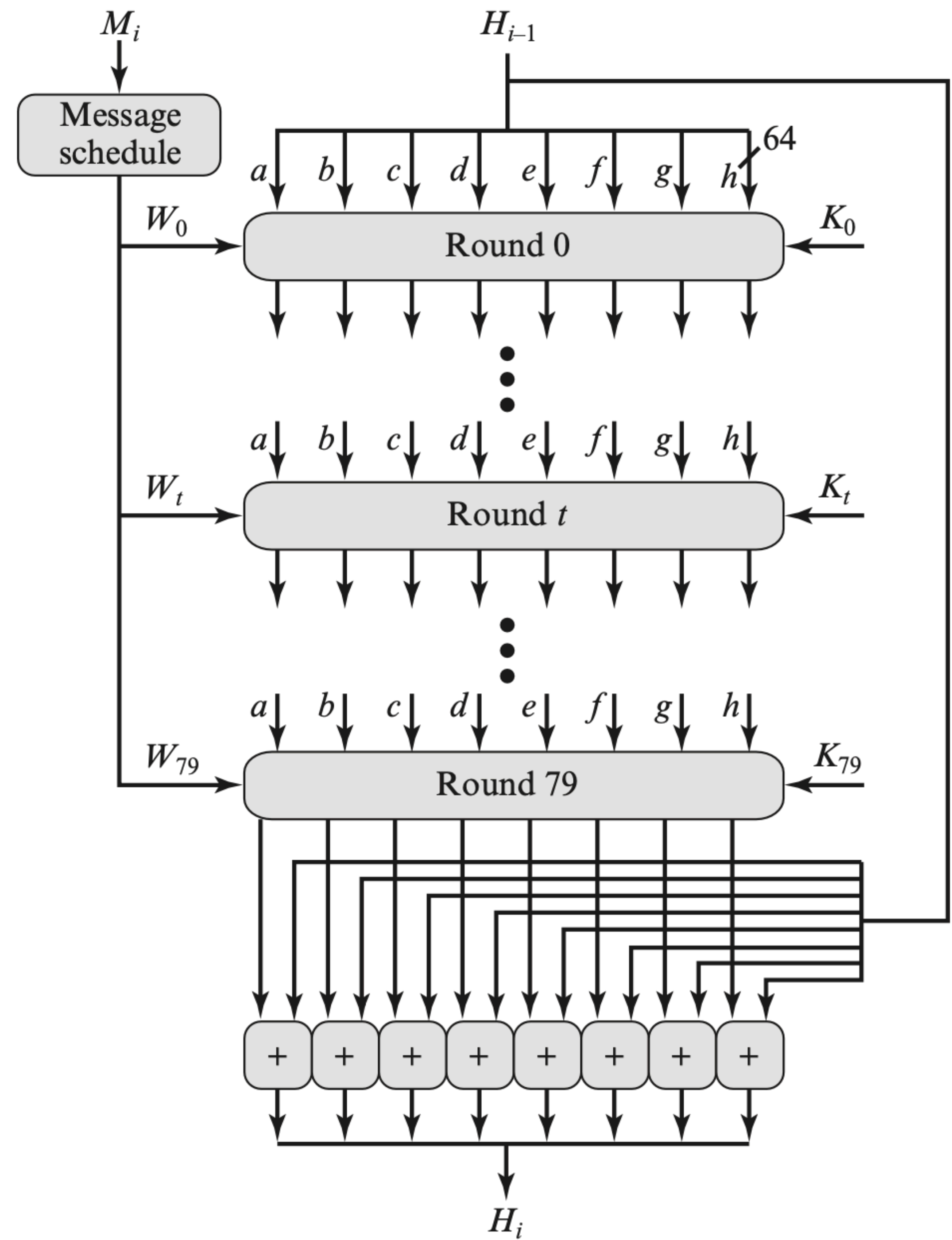
Most important: Never do your own crypto! Please use reference implementations!

Algorithm	Message Size	Block Size	Word Size	Message Digest Size
SHA-1	$< 2^{64}$	512	32	160
SHA-224	$< 2^{64}$	512	32	224
SHA-256	$< 2^{64}$	512	32	256
SHA-384	$< 2^{128}$	1024	64	384
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/224	$< 2^{128}$	1024	64	224
SHA-512/256	$< 2^{128}$	1024	64	256

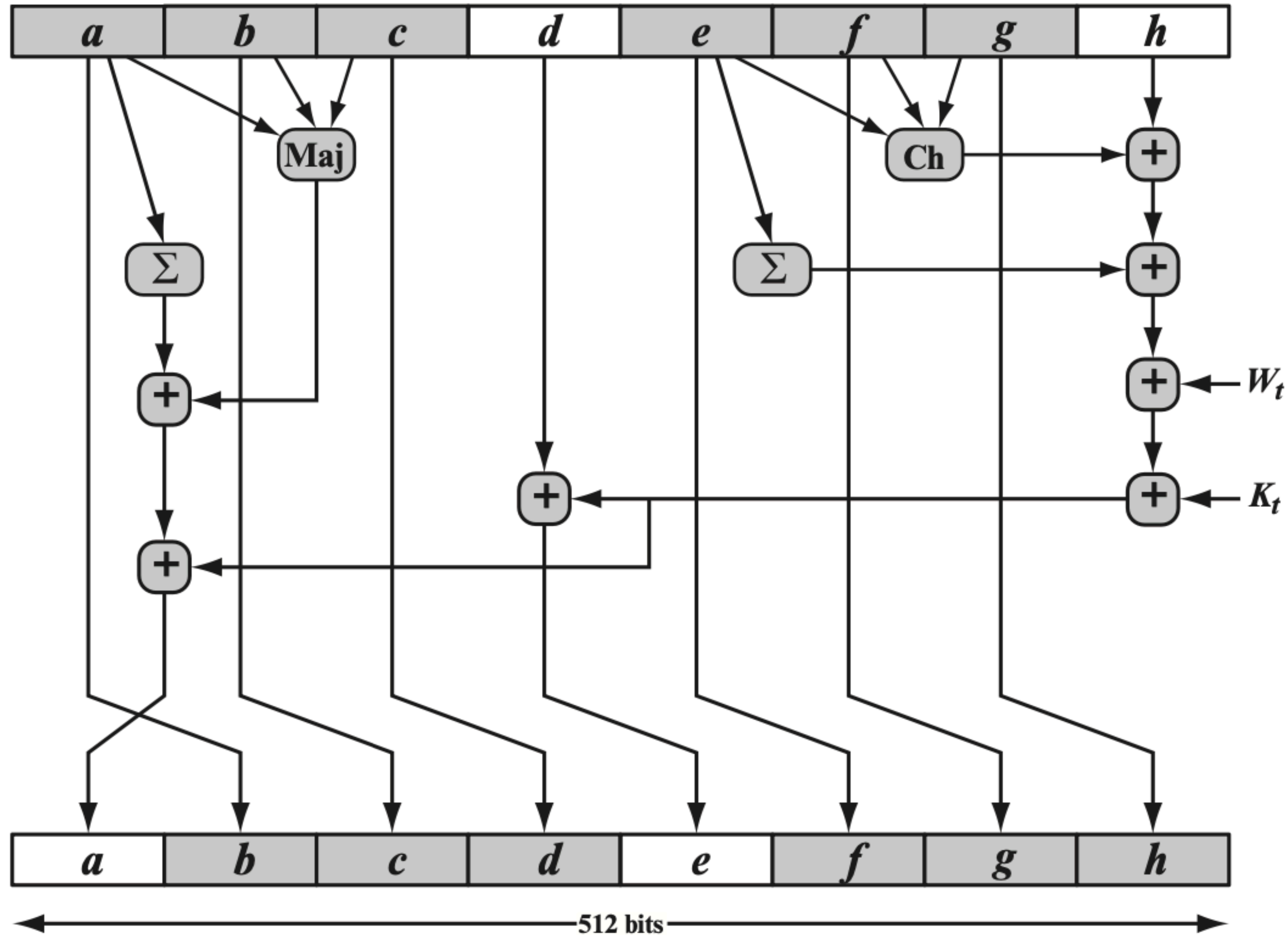
SHA-512 Logic



SHA-512 Logic



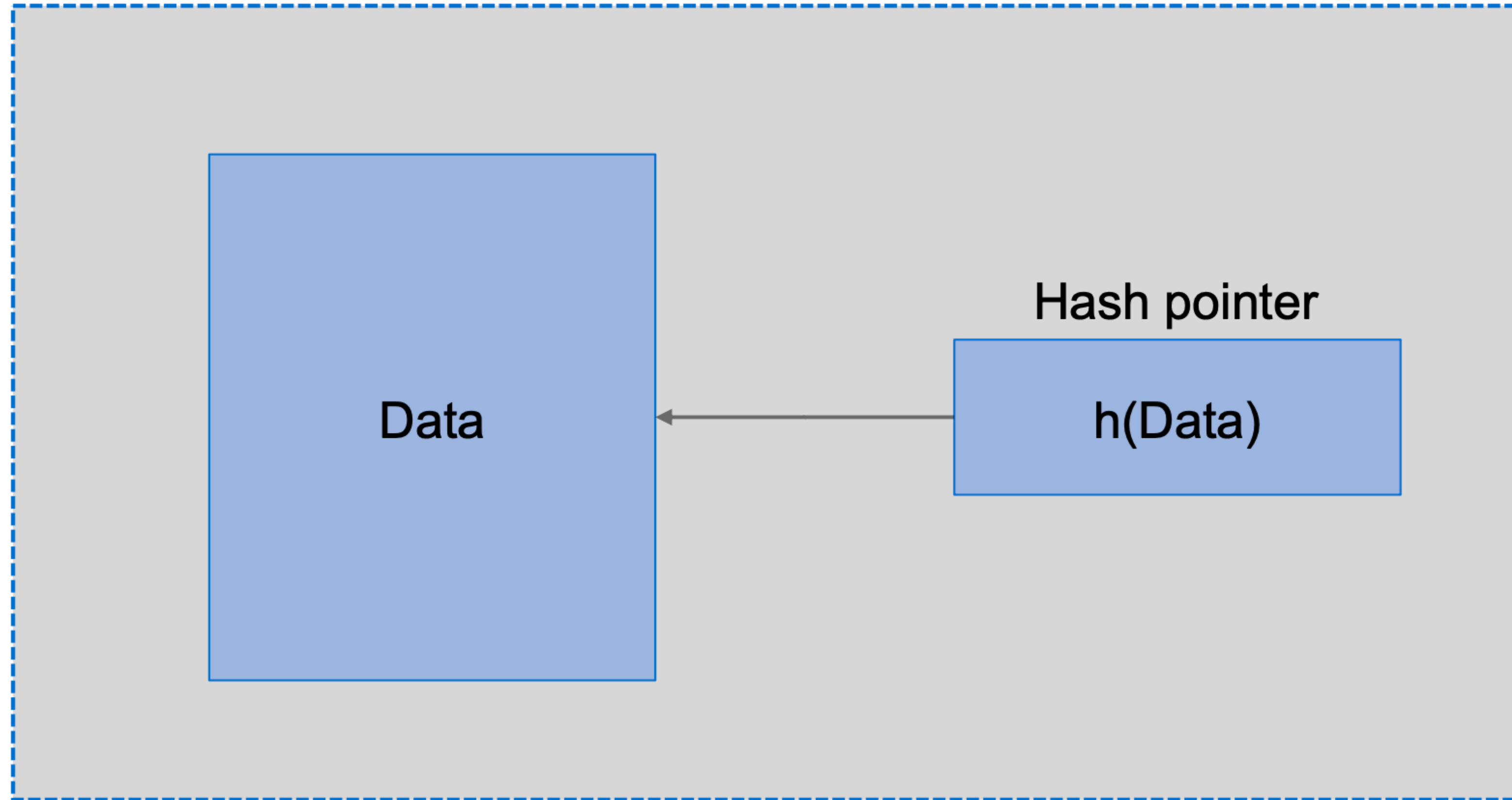
SHA-512 Logic



SHA-256 in Python

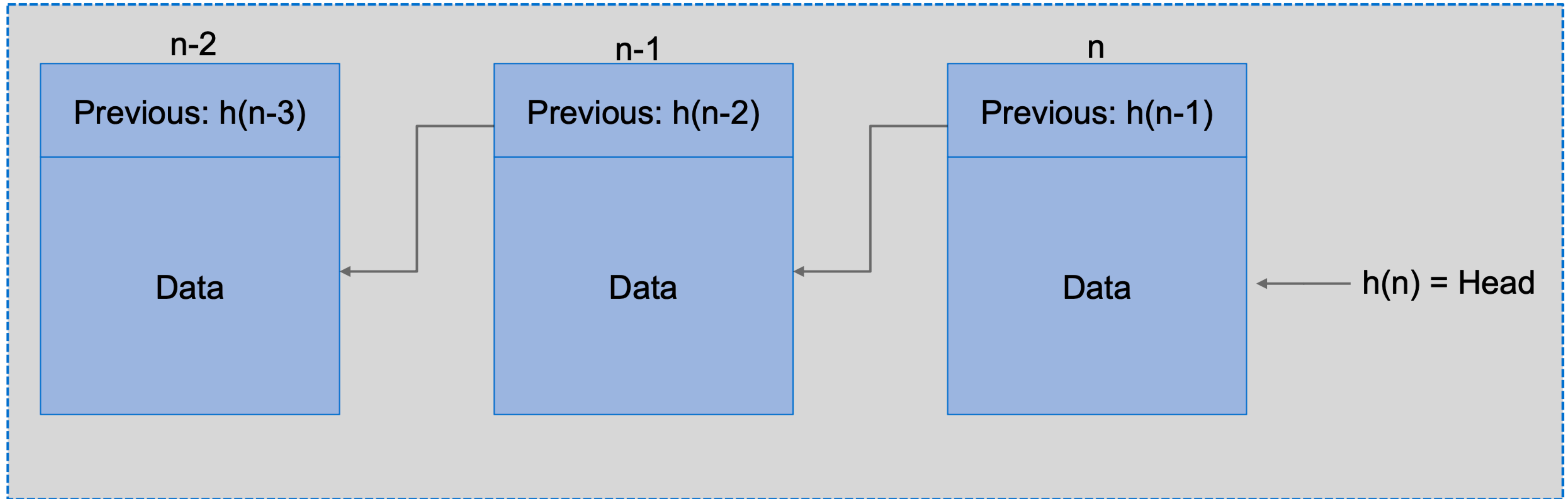
```
import hashlib  
data = "Hello, blockchain!".encode('utf-8')  
hash_value = hashlib.sha256(data).hexdigest()  
print(hash_value)
```

Hash Pointer



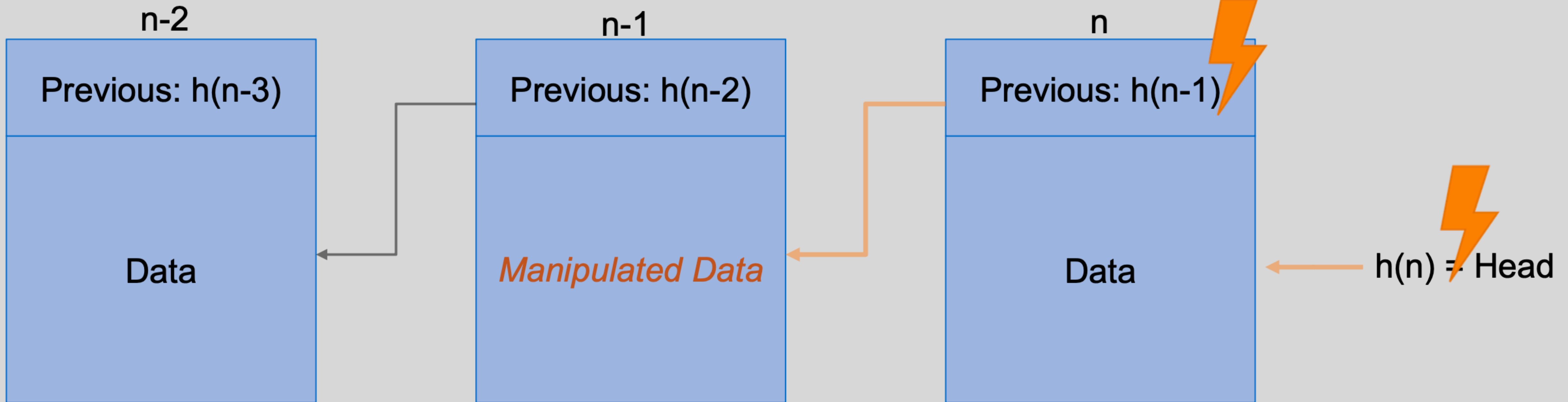
- Cryptographic hash of the original data.
- Points to where data resides (not the location).
- Allows us to verify that the information has not changed.

Linked List of Hash Pointers



- A blockchain can be considered as a **linked list of hash pointers**.
- The hash pointers **ensure the integrity** of the complete blockchain.
- Even if all hashes up to the head are recalculated, detecting the change in head hash would only require storing the correct head hash instead of the whole chain.

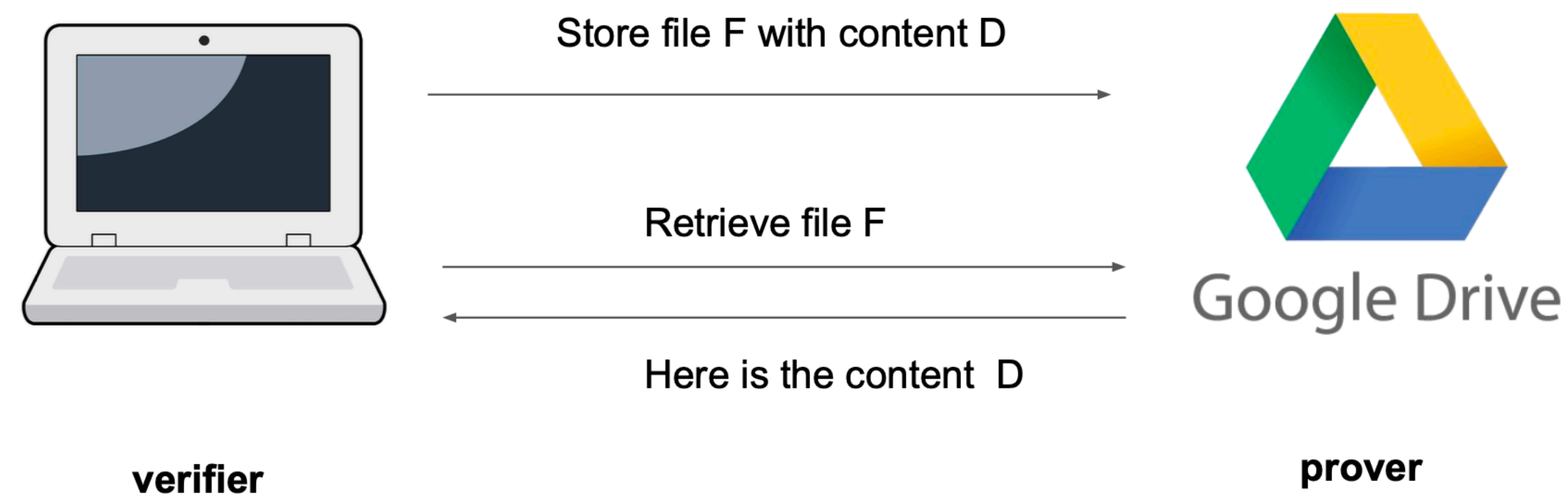
Linked List of Hash Pointers



Data manipulation in block $n-1$ leads to a new head hash!

Merkle Tree

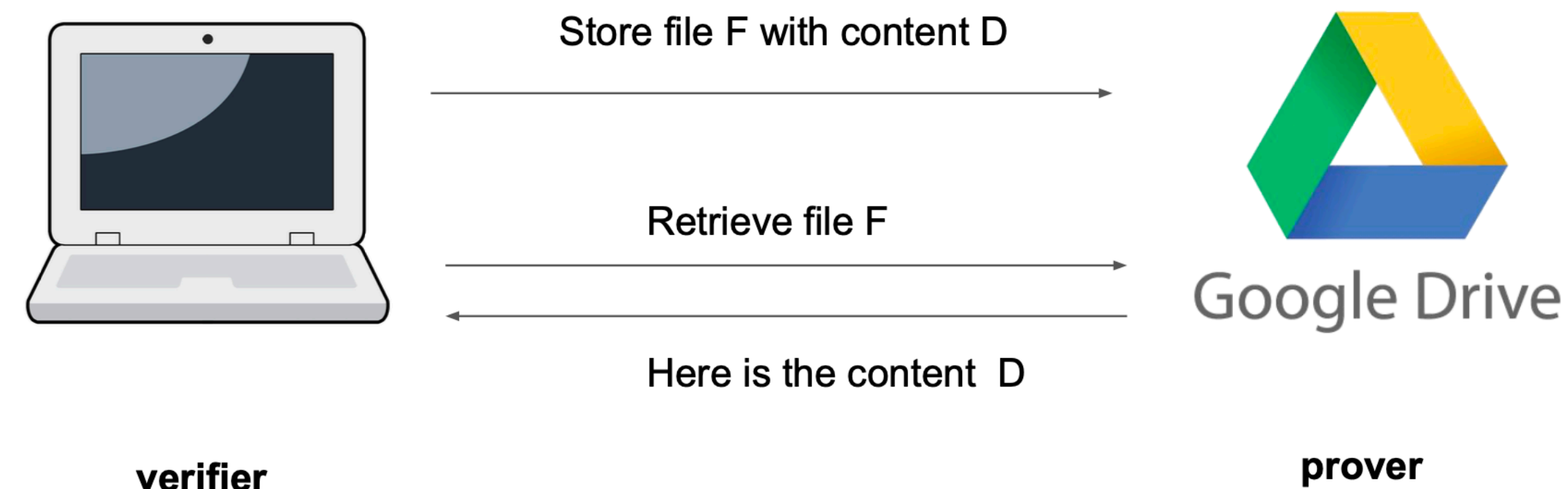
- Like regular data structures, but cryptographically authenticated
- Allows a **verifier** to store, retrieve and operate on data with an untrusted **prover**



- What if server is adversarial and returns $D' \neq D$?

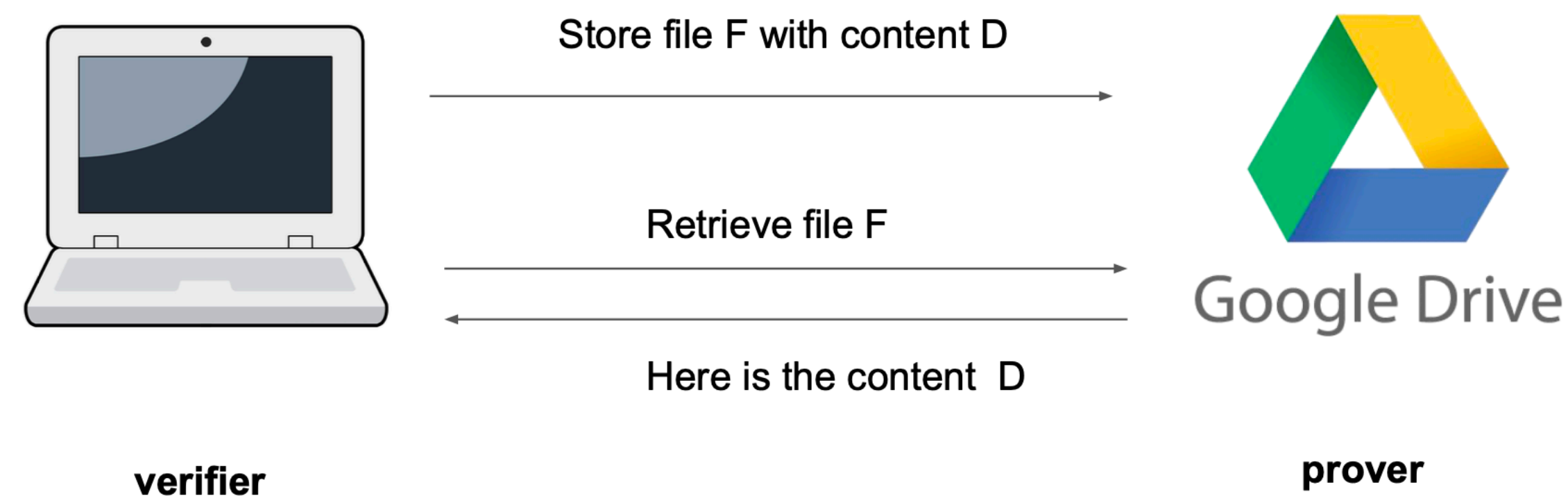
Merkle Tree

- Trivial Solution:
 - Client does not delete D
 - When server returns D' , client compares D and D'
 - What if client doesn't have enough memory to store D for a long time?



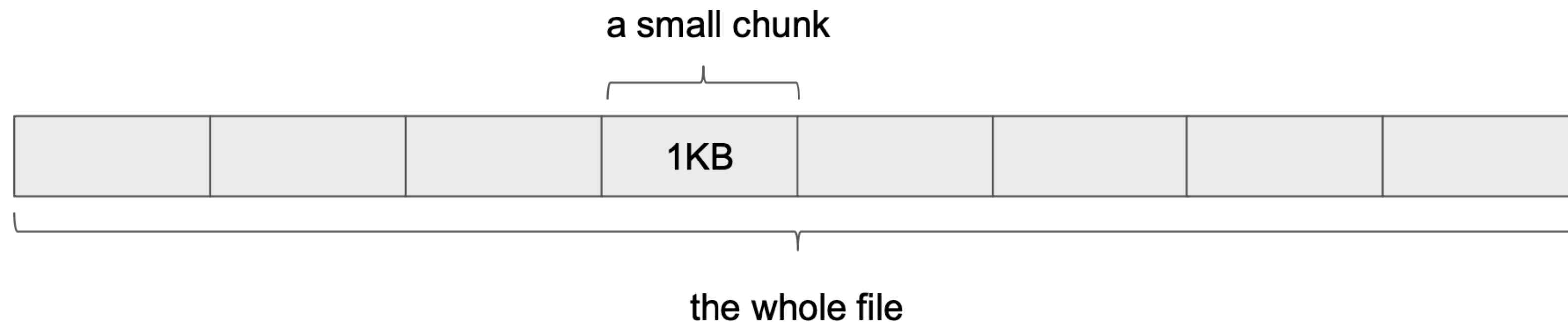
Merkle Tree

- Hashing Solution:
 - Client sends file F with data D to server
 - Server stores (F, D)
 - Client stores $H(D)$, deletes D
 - Client requests F from server
 - Server returns D'
 - Client compares $H(D') = H(D)$



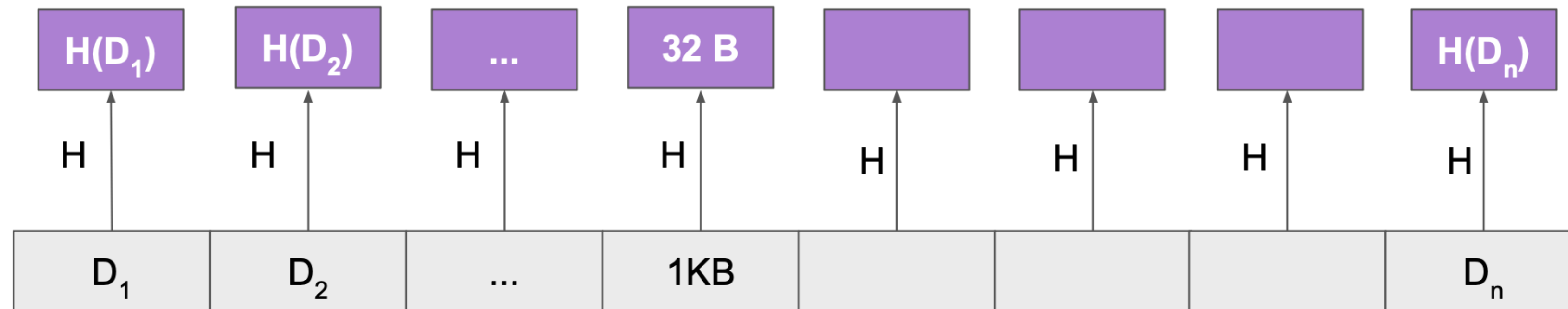
Merkle Tree

- What if client wants to retrieve the 200,019th byte of the file?
 - Must download the whole file...
 - **Merkle trees** to the rescue!



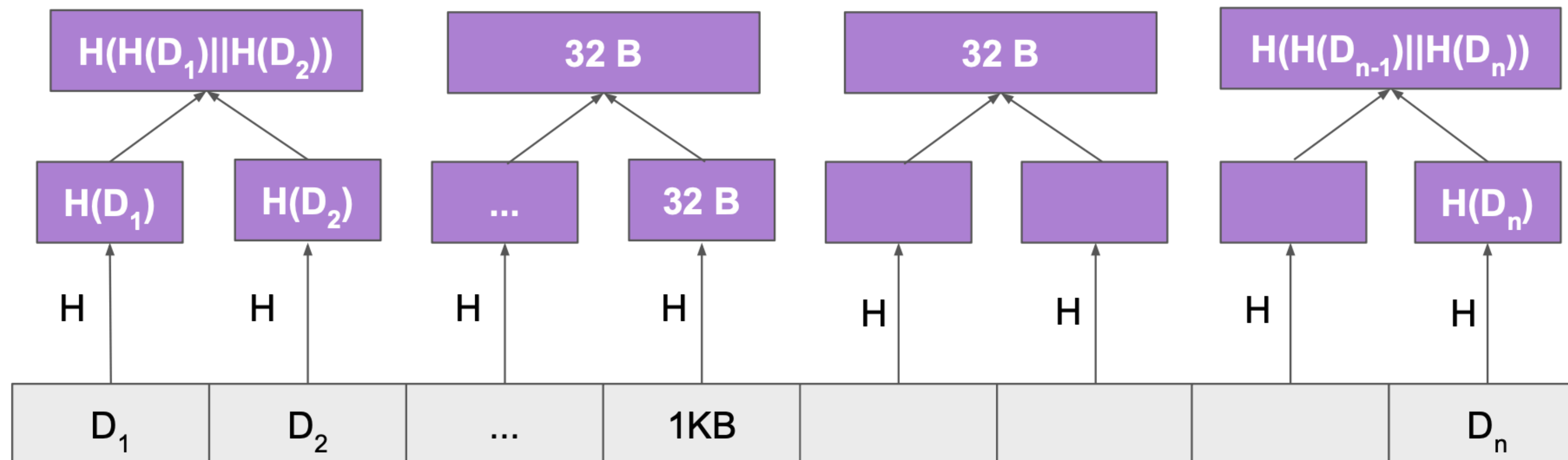
Merkle Tree

- **Hash** each chunk using a cryptographic hash function (SHA256)
- Convention: Arrows show direction of hash function application

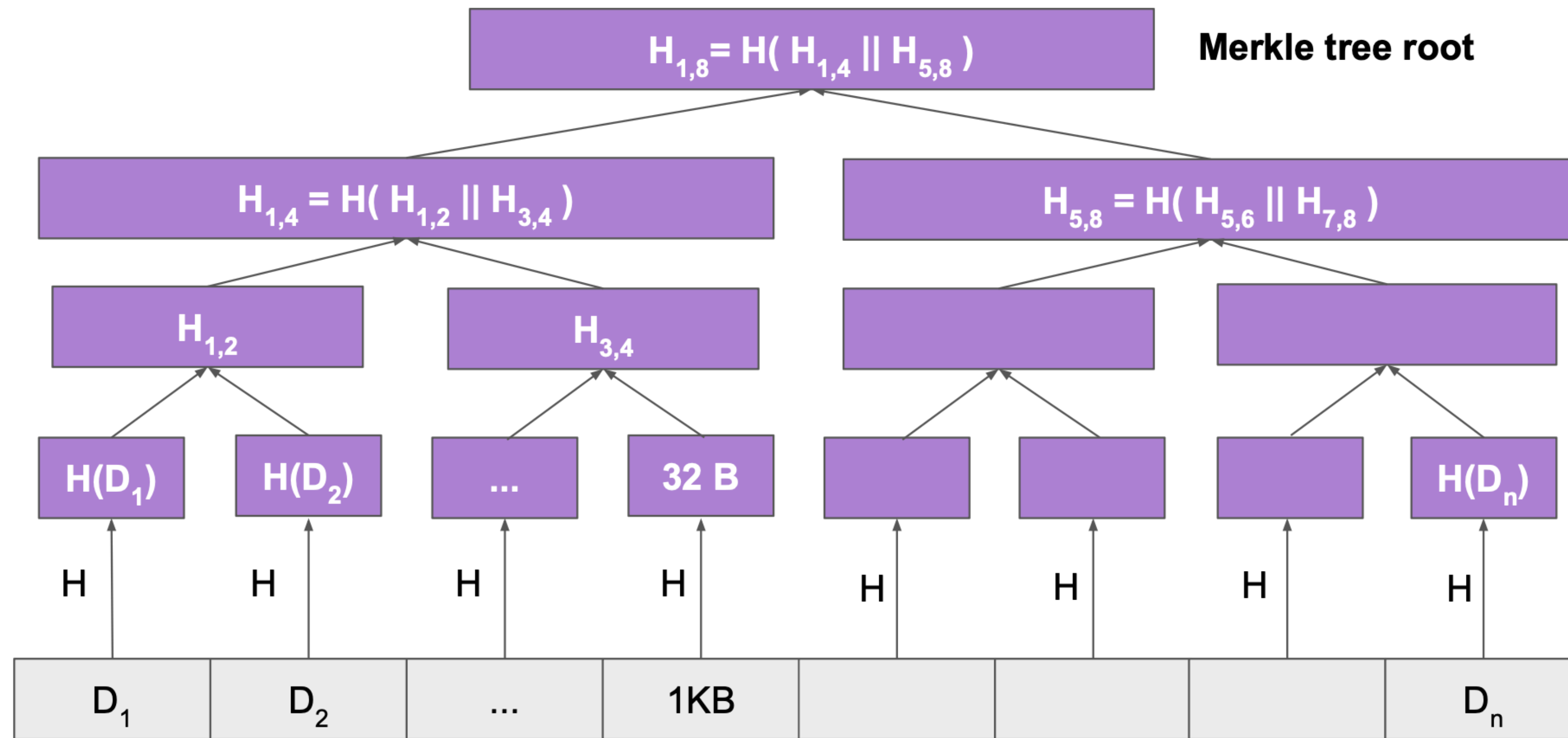


Merkle Tree

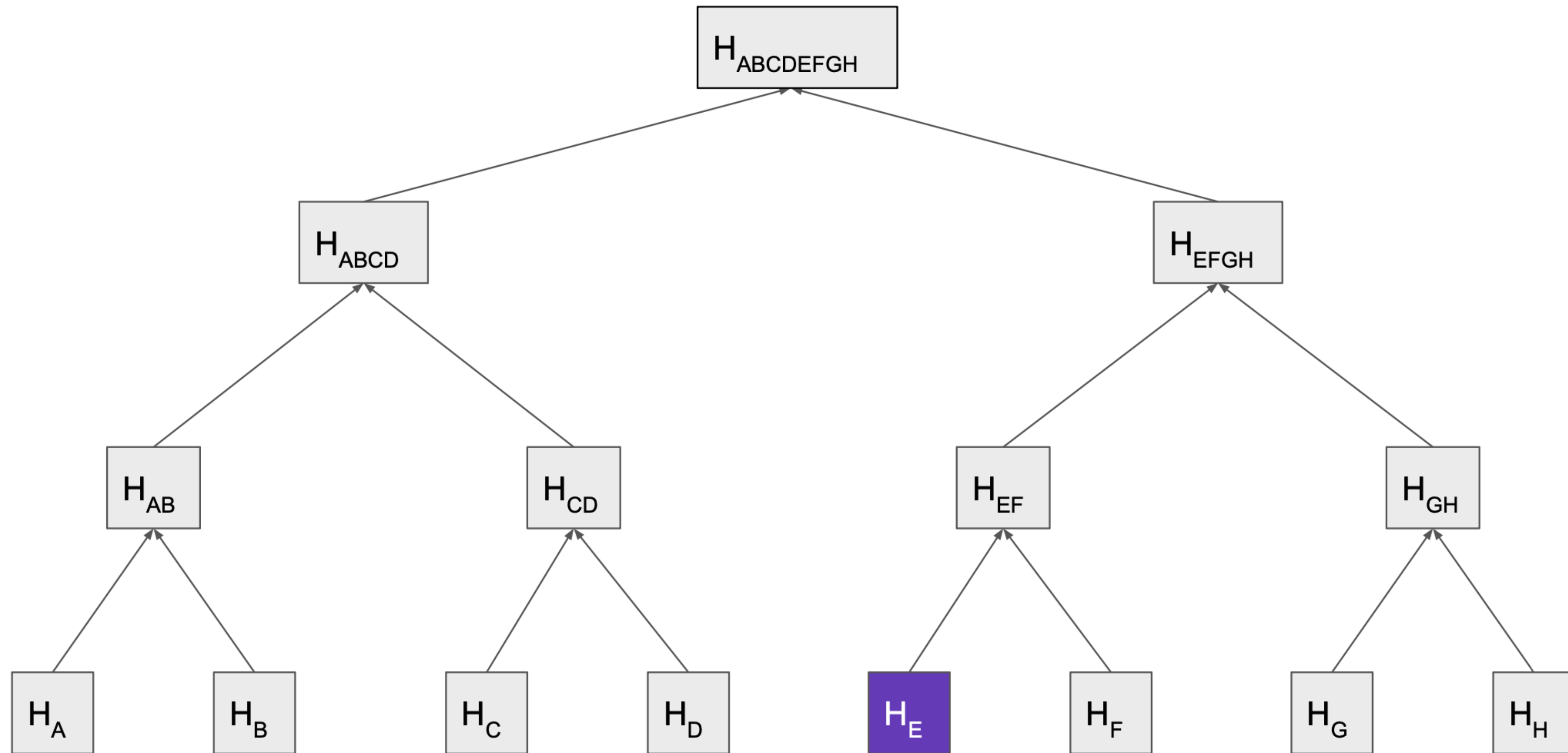
- **Combine** them by two to create a binary tree
- Each node stores the **hash** of the **concat** of its children



Merkle Tree



Merkle Tree



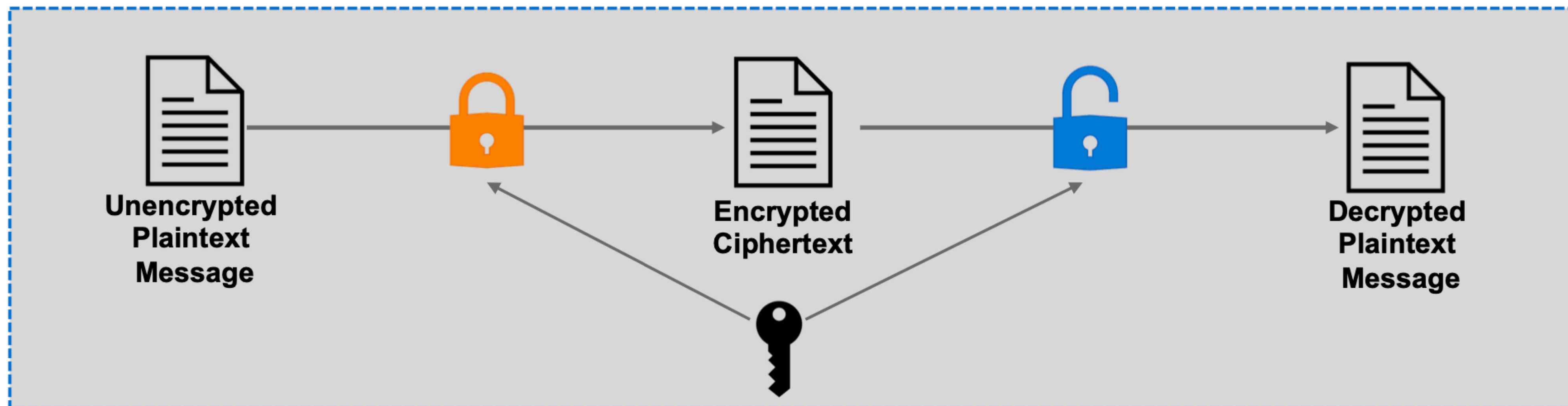
Merkle Tree in Python

```
# Example of building a Merkle tree in Python
def merkle_tree(data):
    if len(data) == 1:
        return data[0]
    new_data = []
    for i in range(0, len(data), 2):
        left = data[i]
        right = data[i + 1] if i + 1 < len(data) else left
        combined_hash = hashlib.sha256(left + right).hexdigest()
        new_data.append(combined_hash)
    return merkle_tree(new_data)
# Usage:
transactions = ["Tx1", "Tx2", "Tx3", "Tx4"]
root_hash = merkle_tree(transactions)
print(root_hash)
```

Symmetric Encryption

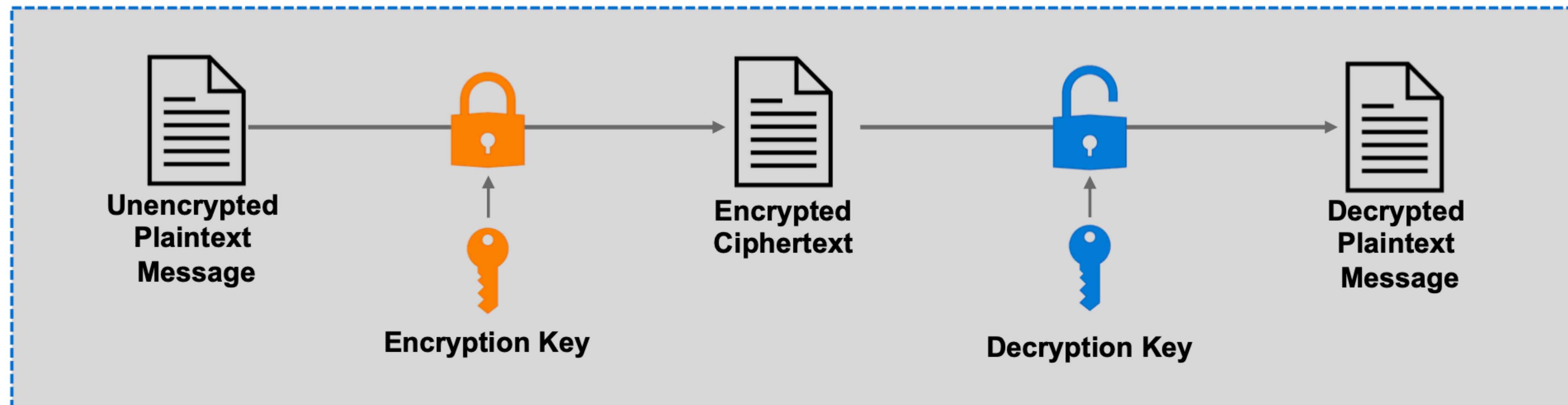
The system is called **symmetric**, in terms of two qualities:

- Encryption and decryption are done using the same secret key.
- The encryption and decryption functions are similar.
- The key must be **exchanged** between the organizations.¹
- With symmetric encryption, data is changed to a form that cannot be understood by anybody who does not have the secret key to decrypt it. When the receiver with the key receives the message, the algorithm reverses its action so that the message is returned to its original.

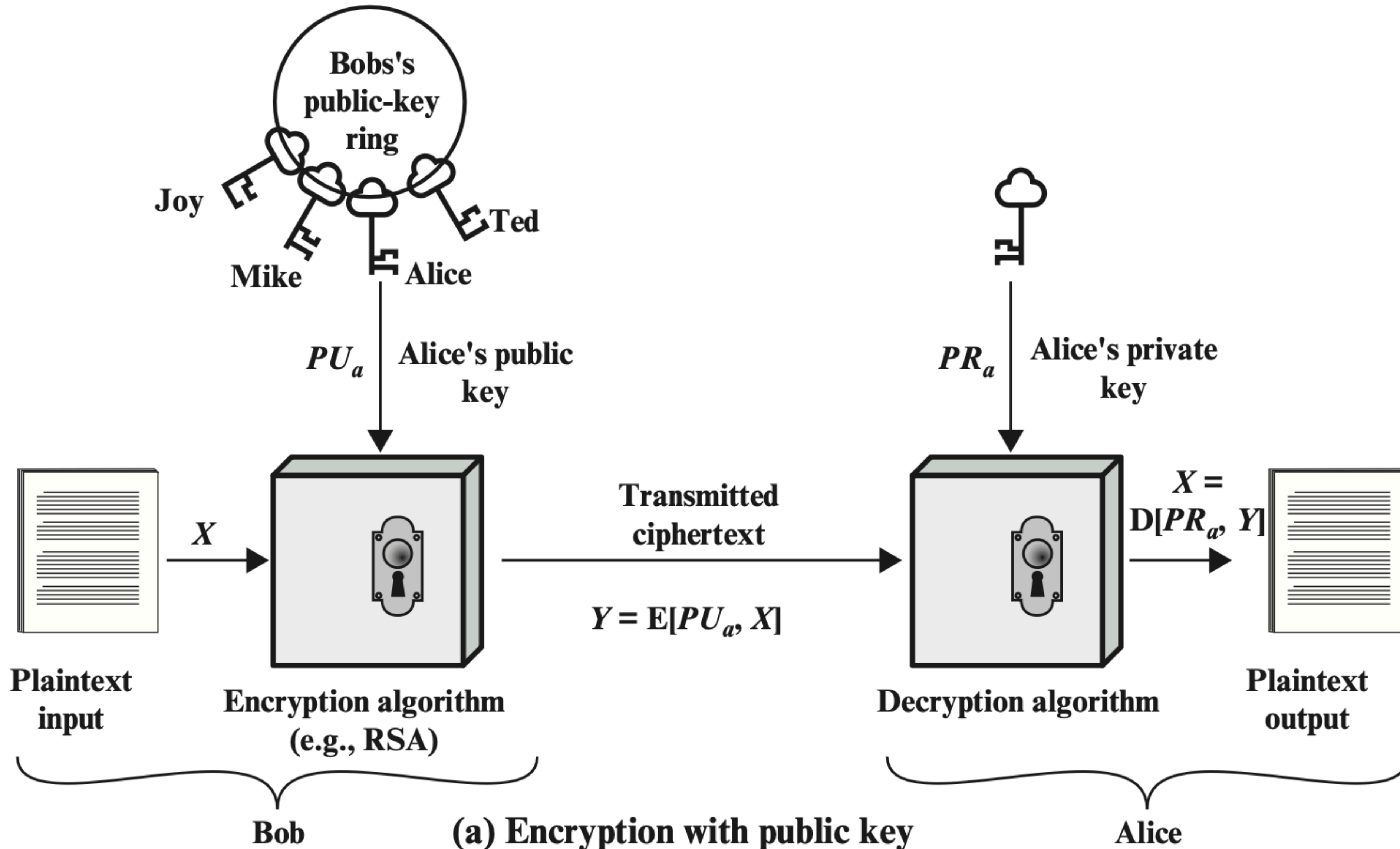


Asymmetric Encryption

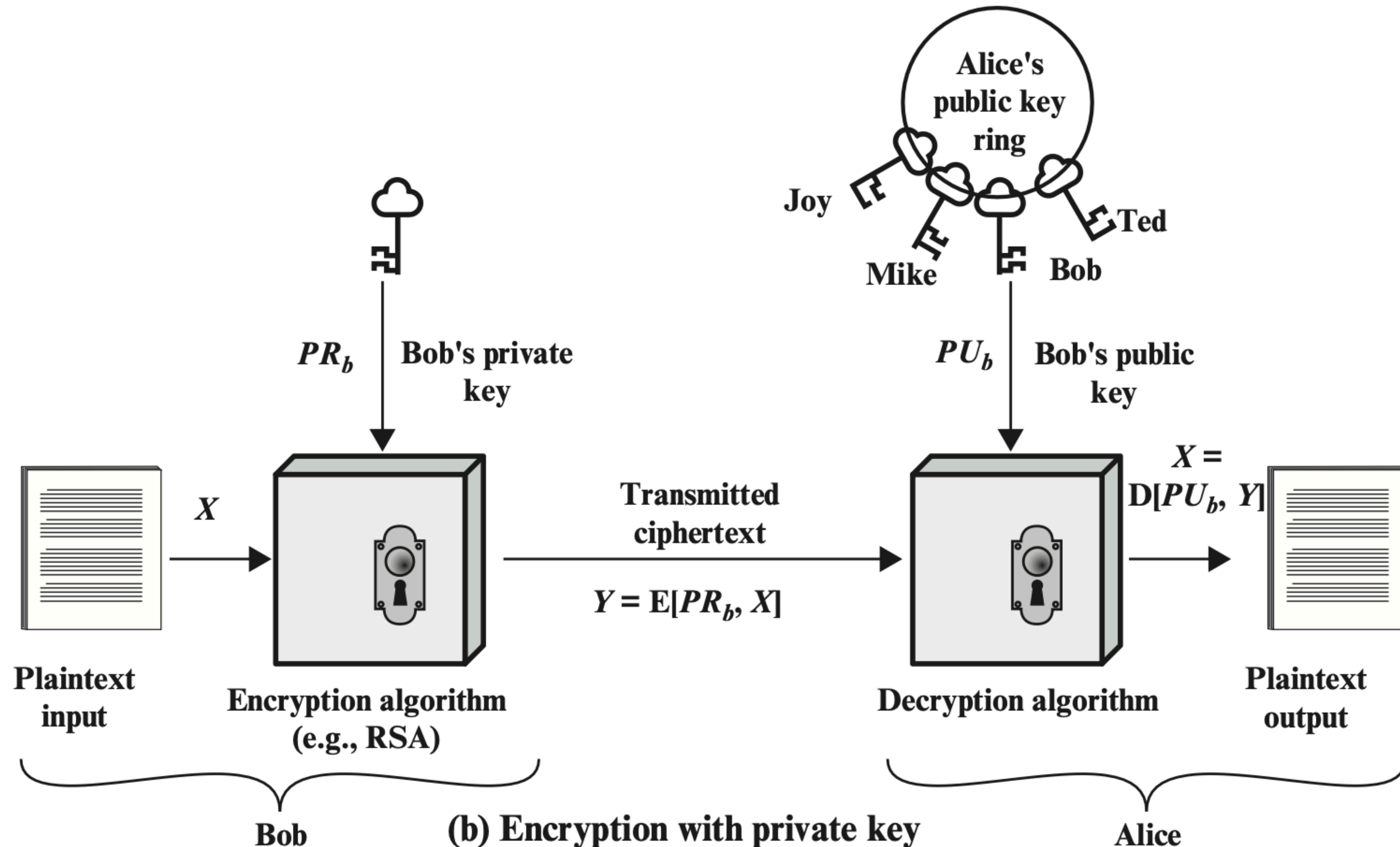
- In **asymmetric** cryptography¹, **pairs of related keys** are used (one **public** and one **private key**). To generate these key pairs, one-way functions are utilized.
- A message receiver publishes a **public encryption key** that is **known to everyone** and also has a **matching private key for decryption**.
- A sender cannot read the messages of another sender, even if both have the receiver's public key, due to the one-way nature of the encryption algorithm.
- It is not essential for the person who encrypts the message to have a secret key. The important part is that **only the receiver can decrypt the message** using a secret key.
- Asymmetric cryptography is often used to authenticate data using digital signatures.



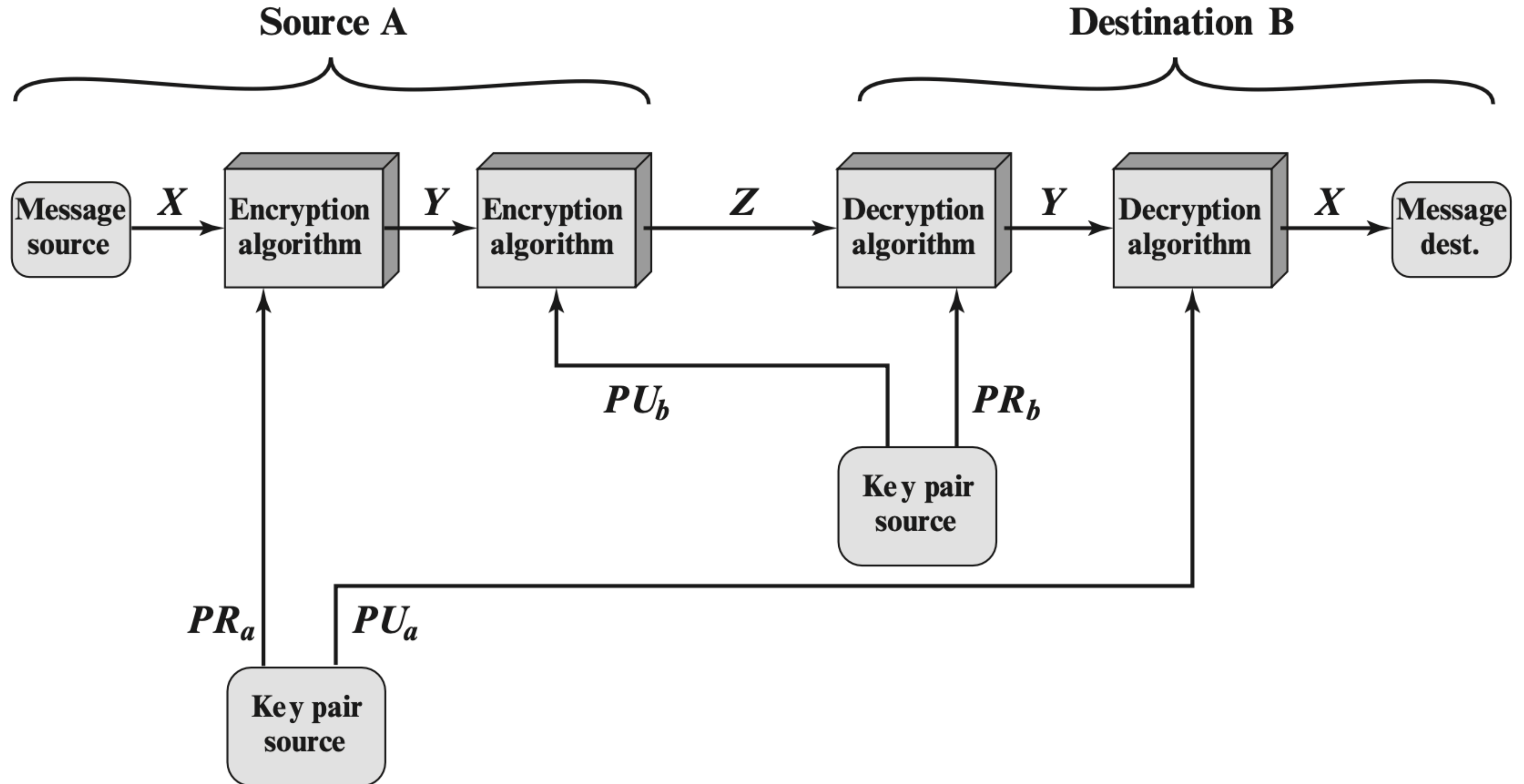
Encryption with Public Key: Confidentiality



Encryption with Public Key: Authentication



Confidentiality and Authentication



Requirements of Asymmetric Encryption

1. It is computationally easy for a party B to generate a key pair (public key PU_b , private key PR_b).
2. It is computationally easy for a sender A, knowing the public key and the message to be encrypted, M , to generate the corresponding ciphertext:

$$C = E(PU_b, M)$$

3. It is computationally easy for the receiver B to decrypt the resulting ciphertext using the private key to recover the original message:

$$M = D(PR_b, C) = D[PR_b, E(PU_b, M)]$$

Requirements of Asymmetric Encryption

4. It is computationally infeasible for an adversary, knowing the public key, PU_b , to determine the private key, PR_b .
5. It is computationally infeasible for an adversary, knowing the public key, PU_b , and a ciphertext, C , to recover the original message, M .

We can add a sixth requirement that, although useful, is not necessary for all public-key applications:

6. The two keys can be applied in either order:

$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)]$$

One-way vs. One-way Trapdoor Functions

- One-way Functions:

$$\begin{array}{ll} Y = f(X) & \text{easy} \\ X = f^{-1}(Y) & \text{infeasible} \end{array}$$

- One-way Trapdoor Functions:

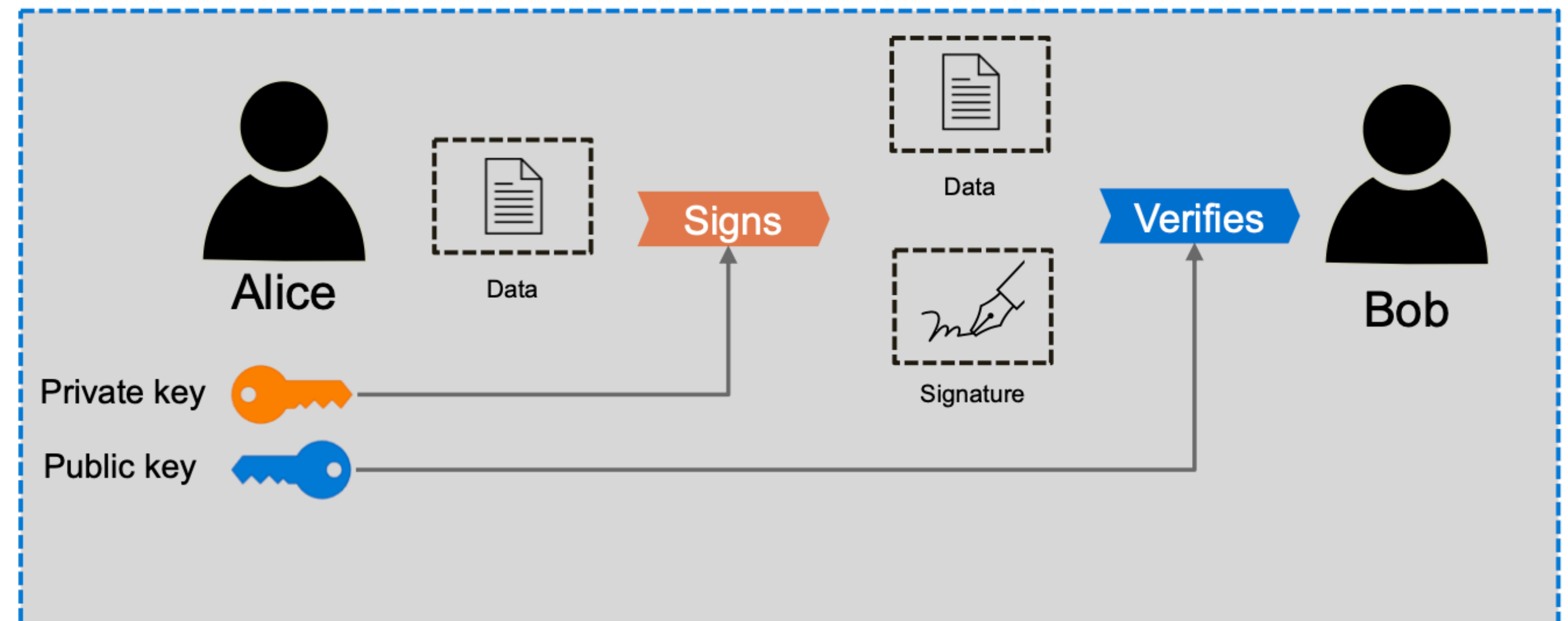
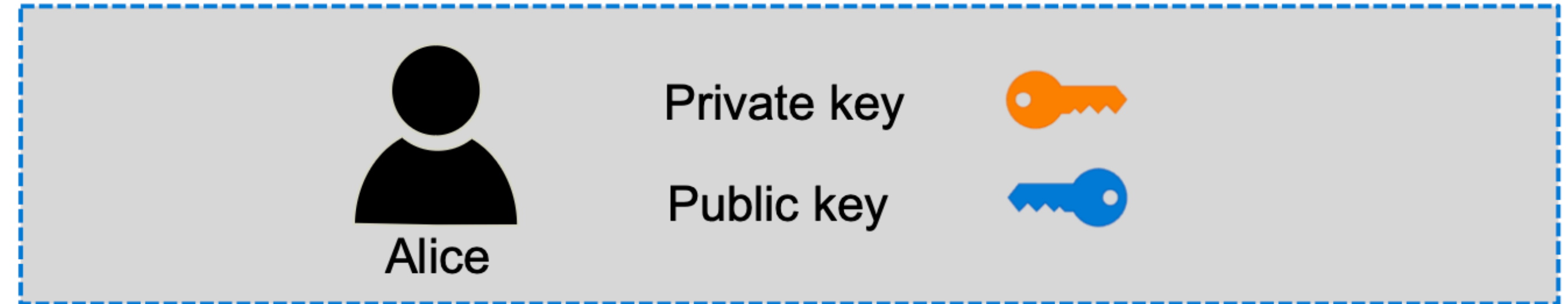
$$\begin{array}{ll} Y = f_k(X) & \text{easy, if } k \text{ and } X \text{ are known} \\ X = f_k^{-1}(Y) & \text{easy, if } k \text{ and } Y \text{ are known} \\ X = f_k^{-1}(Y) & \text{infeasible, if } Y \text{ is known but } k \text{ is not known} \end{array}$$

Digital Signatures

- **RSA**-based signature schemes, such as RSA-PSS
 - Invented in 1977 by Rivest, Shamir and Adleman
 - Based on the assumption that the factorization of large prime number multiplied is very hard, but easy with additional information (so called trapdoor one-way-functions)
 - Long signatures and public keys, fast to verify
 - Not used in blockchains
- **ECC**-based signature schemes, such as ECDSA
 - Suggested independently by Neal Koblitz and Victor S. Miller in 1985
 - Based on discrete logarithms
 - Short signatures and public keys
 - Used by Bitcoin and Ethereum (pre-merge)

Digital Signatures

- A digital signature is a mathematical technique used to validate the authenticity and integrity of a message, software or digital document.
- Digital signatures are based on **cryptographic signature schemes** like RSA and ECC (Due to smaller key sizes in ECC, Bitcoin uses ECDSA).
- We need two properties of (analogue) signatures to hold in the digital world:
 - **Only** an **entity** is able to **create** a **signature** of its own, but **everyone** can **verify** it.
 - This **signature** is **tied to data** that gets signed. A signature cannot be used for different data.



Digital Signatures

Three functions of *digital signature scheme*:

- $(sk, pk) := \text{generateKeys}(keysize)$
 - sk is the secret key and is used to sign messages. pk is the public key and is given to everyone. With the pk , they can verify the signature.
- $sig := \text{sign}(sk, message)$
 - The sign method takes the *message* and the secret key, sk , as input and returns a signature for *message* under sk .
- $isValid := \text{verify}(pk, message, sig)$
 - The verify method takes a *message*, a *signature*, and a *public key* as input. It will return *true* if the signature was generated out of the message and the secret key, otherwise *false*.
- Such that $\text{verify}(pk, message, \text{sign}(sk, message)) == \text{true}$ and **signatures are unforgeable**.

Unforgeability:

- The attacker knows your public key pk .
- The attacker sees your signature sig on an arbitrary amount of *messages*.
- Unforgeable means, that the attacker **is not able** to create a signature on a message that he has not seen.

Digital Identity

- Digital Signature Schemes can be used as **identity systems**
 - The public key ***pk*** acts as an **identity**
 - The private key ***sk*** is the **password** to this identity to act on behalf of this identity
- This has some **advantages**:
 - **New identities** can be generated at will with **generateKeys** from our digital signature scheme
 - At first, these new identities cannot be used to uncover your real-world identity¹
- Additionally:
 - You want to **hash** your public key *pk* in order to receive an “identity”, as
 - Public keys are very large
 - Public keys can be vulnerable to quantum computing attacks²
- To validate a statement, one has to check
 1. if the *pk* hashes to the identity and
 2. if the message verifies under the public key *pk*.

Practical Concerns:

- The **private keys** are **not recoverable**. Once the file is lost, there is no way to act under this entity, can result in lost money, assets, or more.
- An **appropriate key length** should be considered. If the key length is too short, it could be computed in the future.

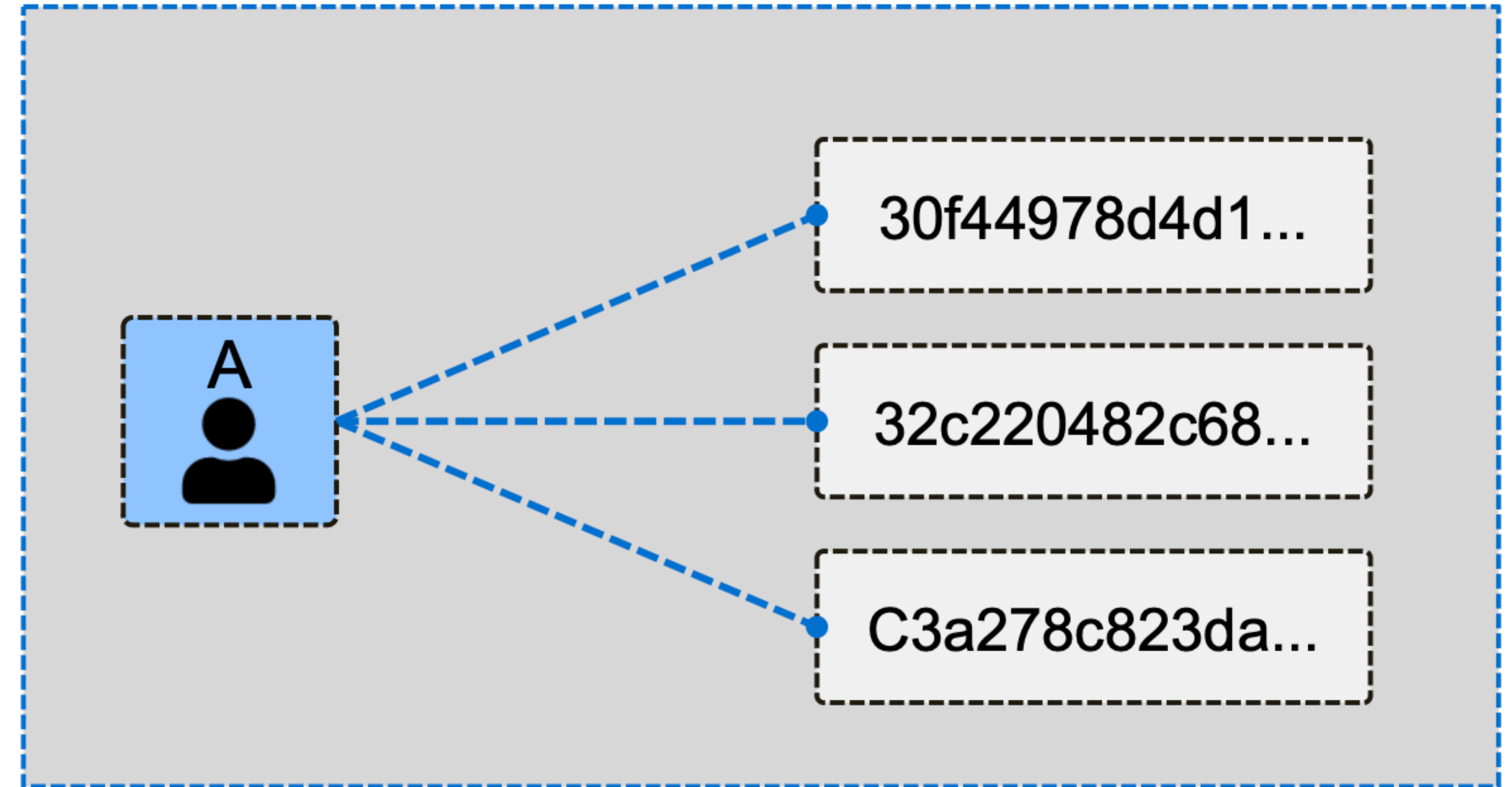
¹ Your statements may leak information, allowing to connect your real world identity to *pk*. *You are pseudonymous.*

Digital Identity

This approach enables a decentralized identity management

- No need for registering at a central authority
- **Arbitrary amount** of identities
- **Simple verification**

All cryptocurrencies / blockchain-based systems handle it this way.



The **address** is (in Ethereum) the **hash of a public key**.

Ethereum



Vitalik Buterin at a Techcrunch conference

- In **November 2013**, Vitalik Buterin started working on the **first version** of the Ethereum **white paper**
- Buterin made the **first public announcement** of Ethereum on **24th of January 2014** at the Bitcoin conference in Miami
- On the **7th of July 2014**, Buterin **announced the start** of the **public crowd sale**
- The **sale lasted 42 days** until 2nd of September
 - For the first 14 days the price was 1 BTC for 2000 ETH
 - After that period the price went up to 1 BTC for 1337 ETH
- In total, **~60 million Ether** were sold in exchange for **31,591 Bitcoins**
 - Worth around 18.5 million USD at that time
 - Used by the Ethereum foundation

Ethereum: Yellow Paper

ETHEREUM: A SECURE DECENTRALISED GENERALISED TRANSACTION LEDGER

EIP-150 REVISION

DR. GAVIN WOOD
FOUNDER, ETHEREUM & ETHCORE
GAVIN@ETHCORE.IO

ABSTRACT. The blockchain paradigm when coupled with cryptographically-secured transactions has demonstrated its utility through a number of projects, not least Bitcoin. Each such project can be seen as a simple application on a decentralised, but singleton, compute resource. We can call this paradigm a transactional singleton machine with shared-state.

Ethereum implements this paradigm in a generalised manner. Furthermore it provides a plurality of such resources, each with a distinct state and operating code but able to interact through a message-passing framework with others. We discuss its design, implementation issues, the opportunities it provides and the future hurdles we envisage.

1. INTRODUCTION

With ubiquitous internet connections in most places of the world, global information transmission has become incredibly cheap. Technology-rooted movements like Bitcoin have demonstrated, through the power of the default, consensus mechanisms and voluntary respect of the social contract that it is possible to use the internet to make

information is often lacking, and plain old prejudices are difficult to shake.

Overall, I wish to provide a system such that users can be guaranteed that no matter with which other individuals, systems or organisations they interact, they can do so with absolute confidence in the possible outcomes and how those outcomes might come about.

Ethereum yellow paper: <https://ethereum.github.io/yellowpaper/paper.pdf>

- **Published in April 2014 by Dr. Gavin Wood**
- **Dr. Gavin Wood is still listed as the only author**
- **Defines the technical specification of Ethereum**
- **Very detailed, contains mathematical function definitions and byte code mappings**
- **Required to implement a full node**
- **Only updated when errors are found or the specification changes**

Ethereum: Foundation

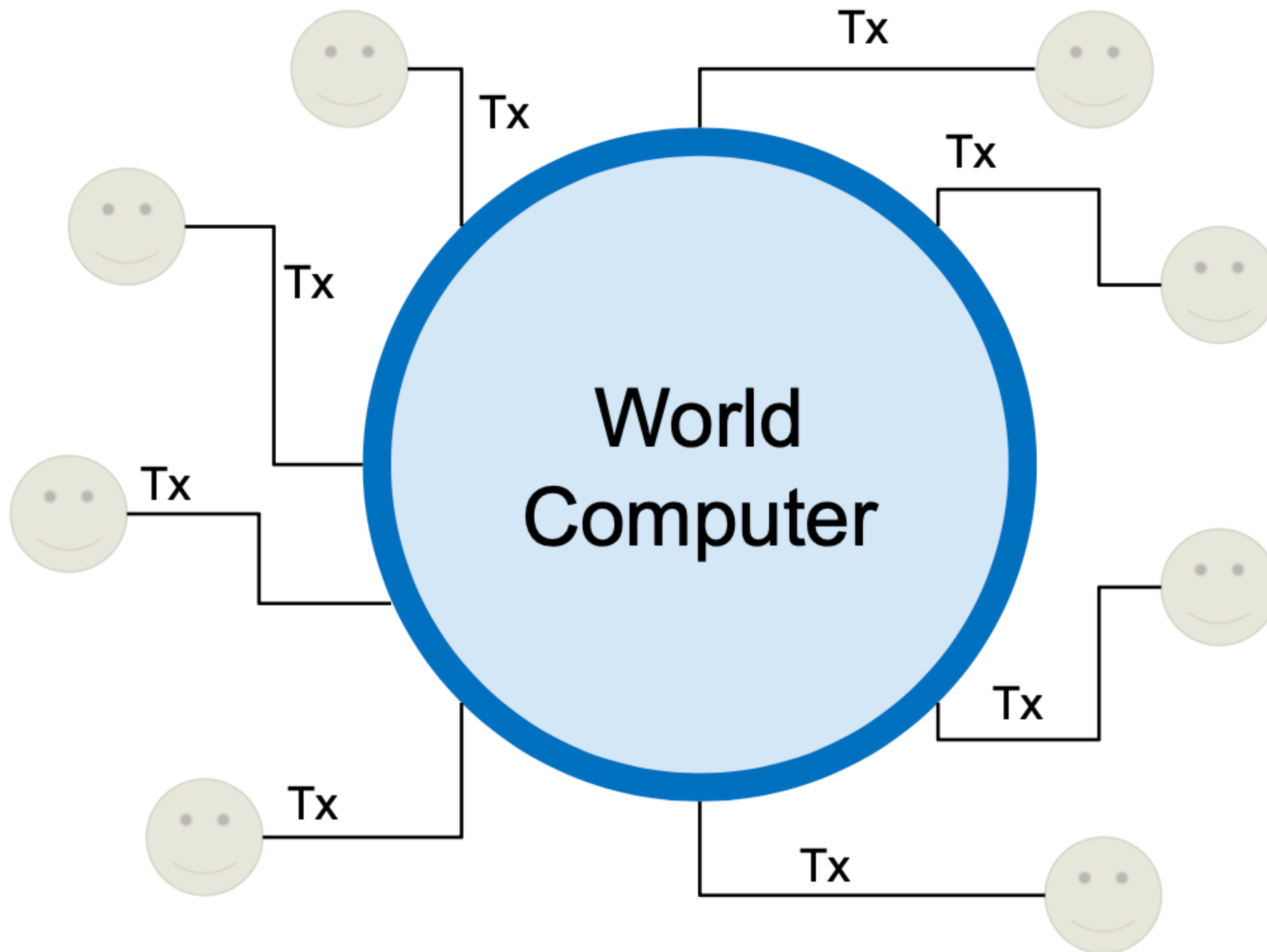
*“The Ethereum Foundation’s **mission** is to **promote and support Ethereum platform and base layer research, development and education** to bring decentralized protocols and tools to the world that empower developers **to produce next generation decentralized applications** (dapps), and together build a more globally accessible, more free and more trustworthy Internet.”*



**ethereum
foundation**

- **Founded in June 2014 in Zug, Switzerland**
- **Non-profit** organization
- **Foundation council** consists of **Vitalik Buterin** and **Patrick Storchenegger** who is responsible for all legal affairs
- **Owns** (or had owned) at least **31,591 Bitcoins** funding capital due to the crowdsale

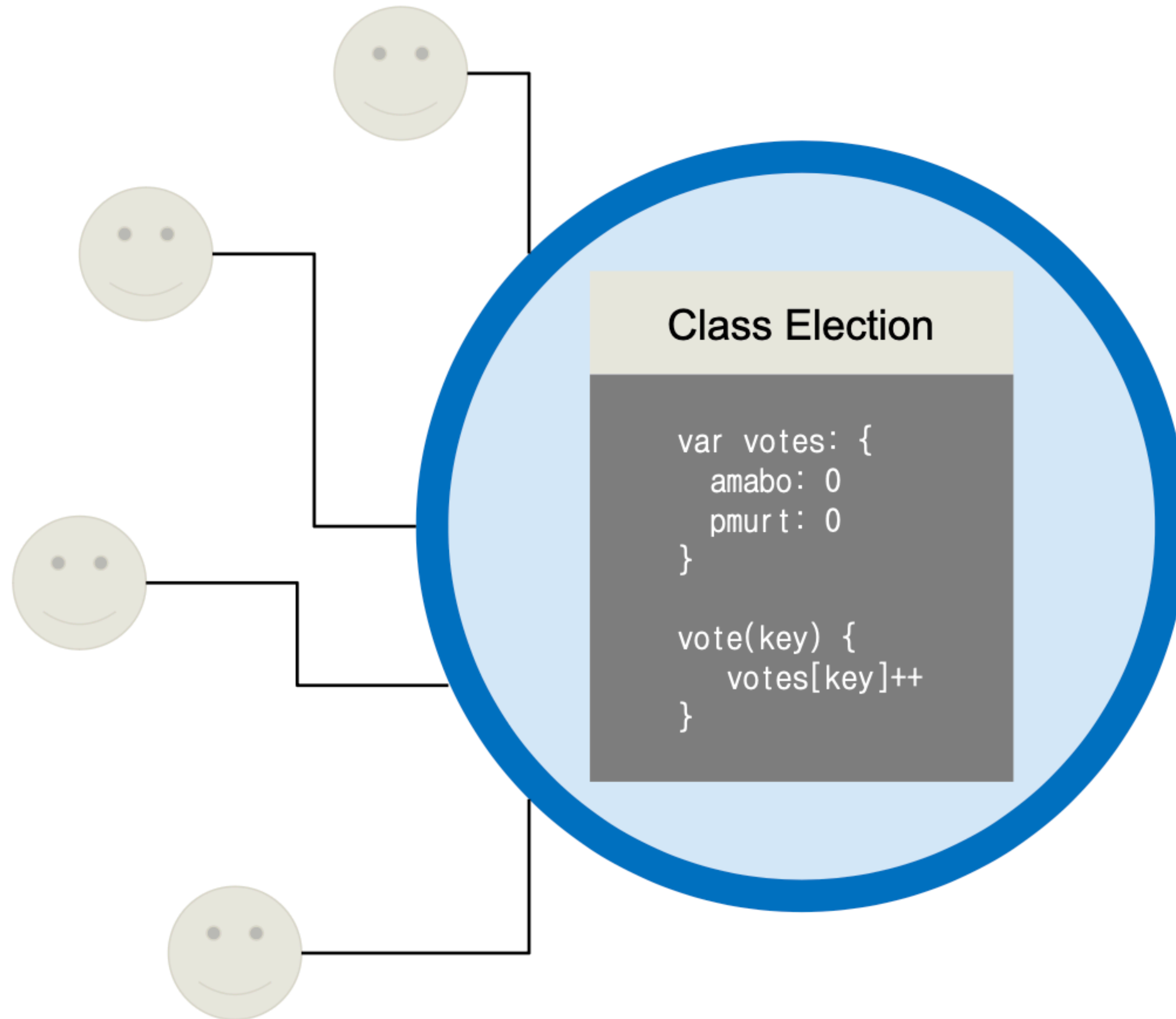
Ethereum: State Machine



Properties

- **All participants are using the same computer**
- **Users issue transactions to call programs on the computer**
- **Everyone shares the same resources and storage**
- **The computer has no explicit, single owner**
- **Using the computer's resources costs money**

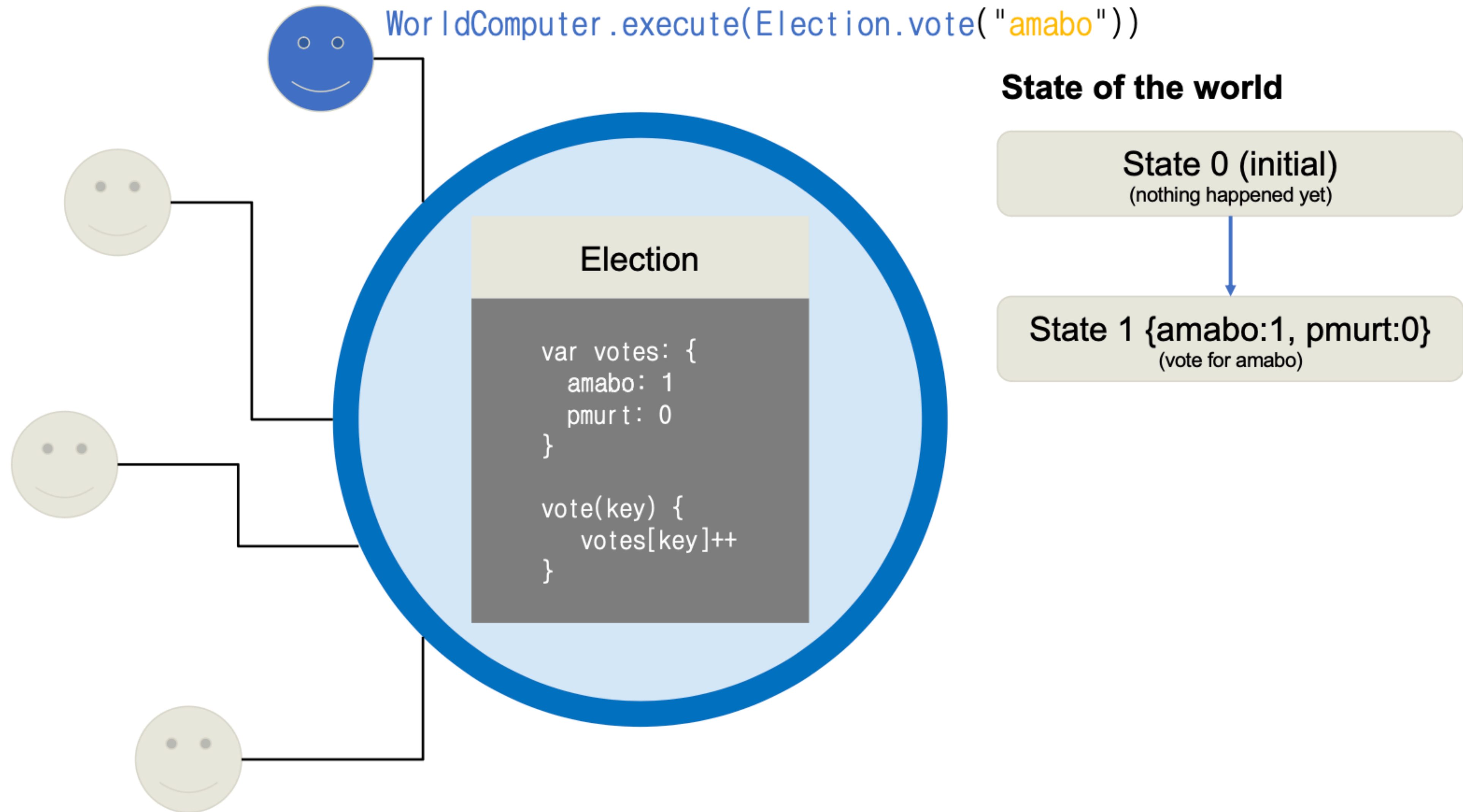
Ethereum: State Machine



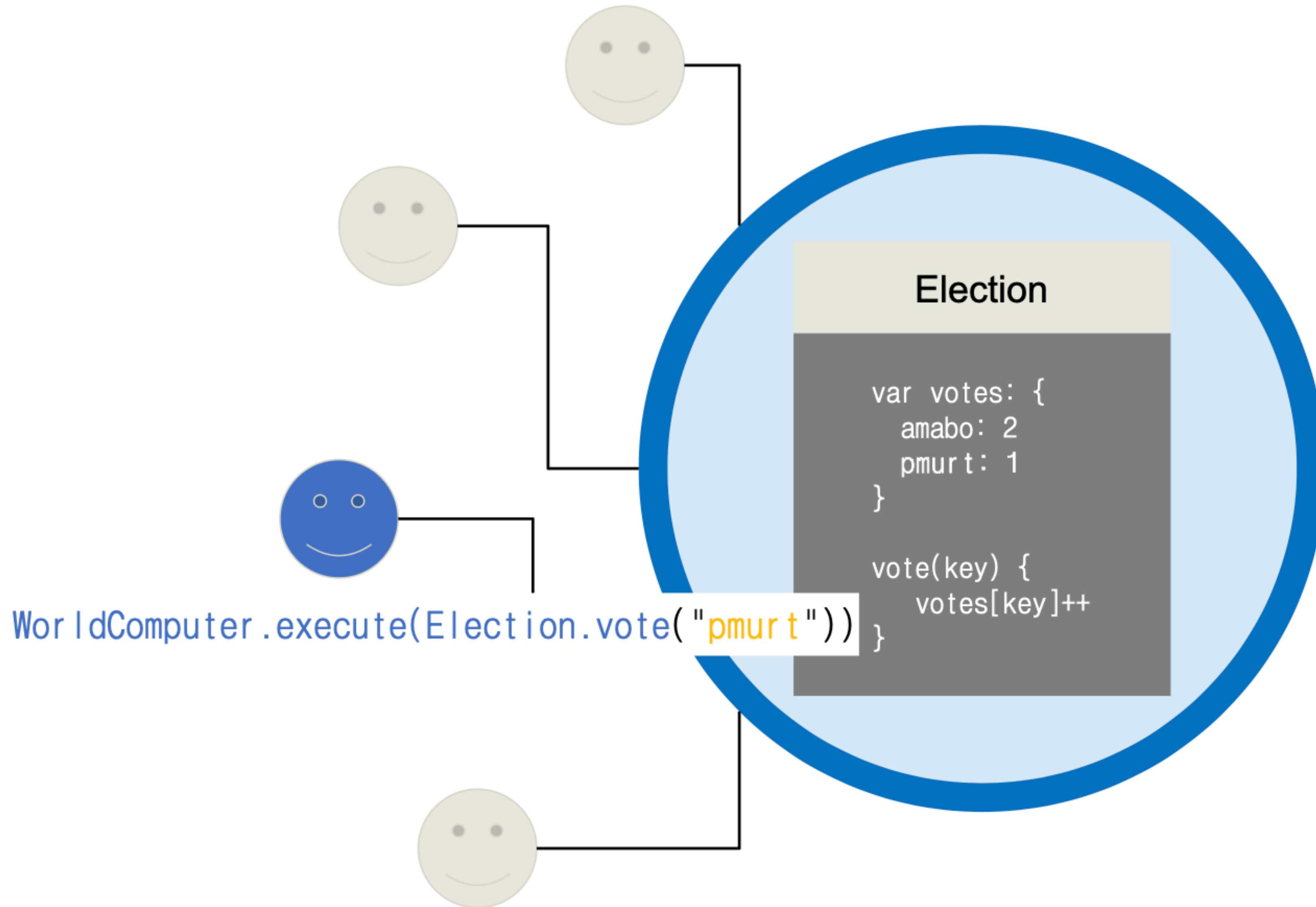
State of the world

State 0 (initial)
(nothing happened yet)

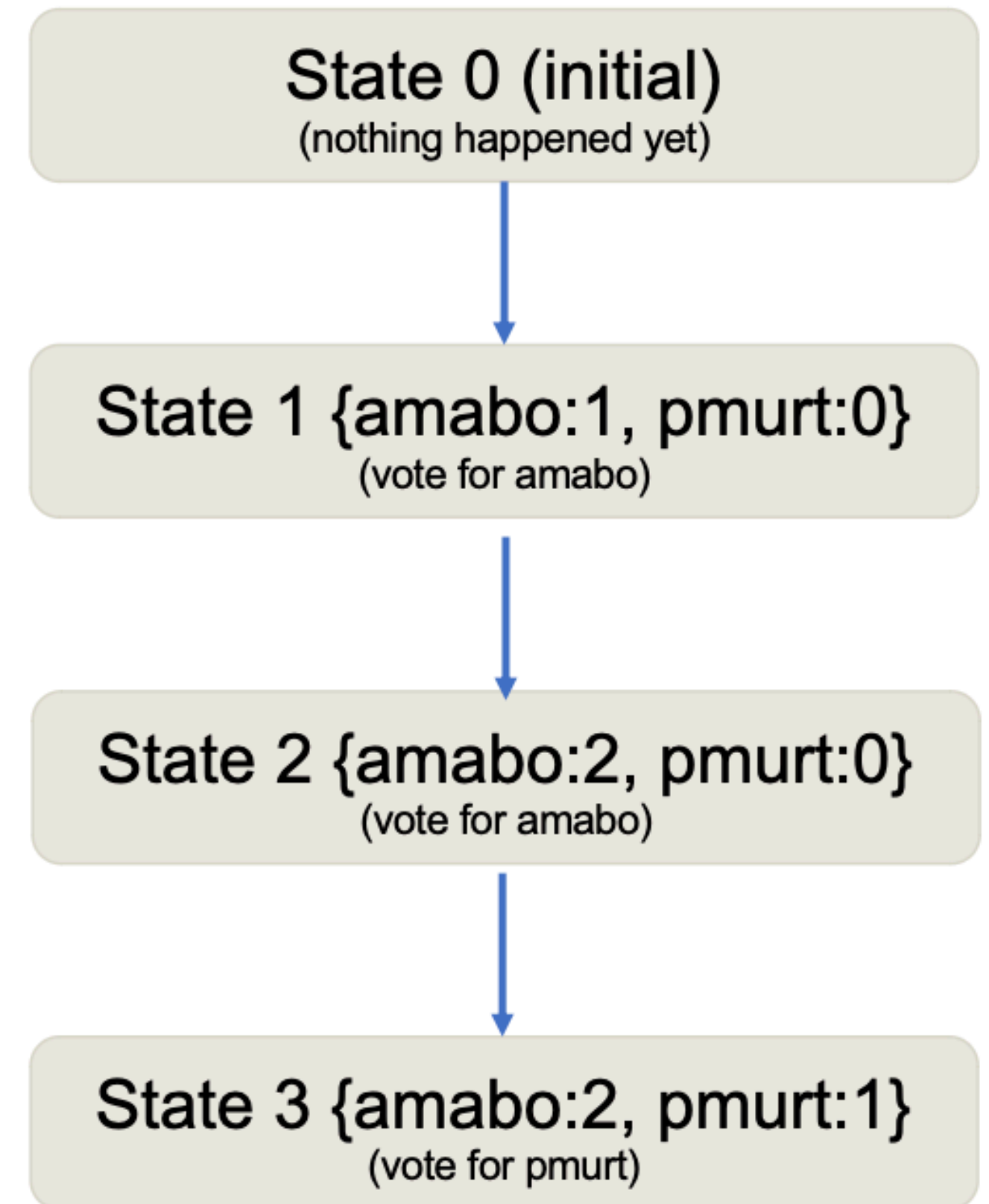
Ethereum: State Machine



Ethereum: State Machine

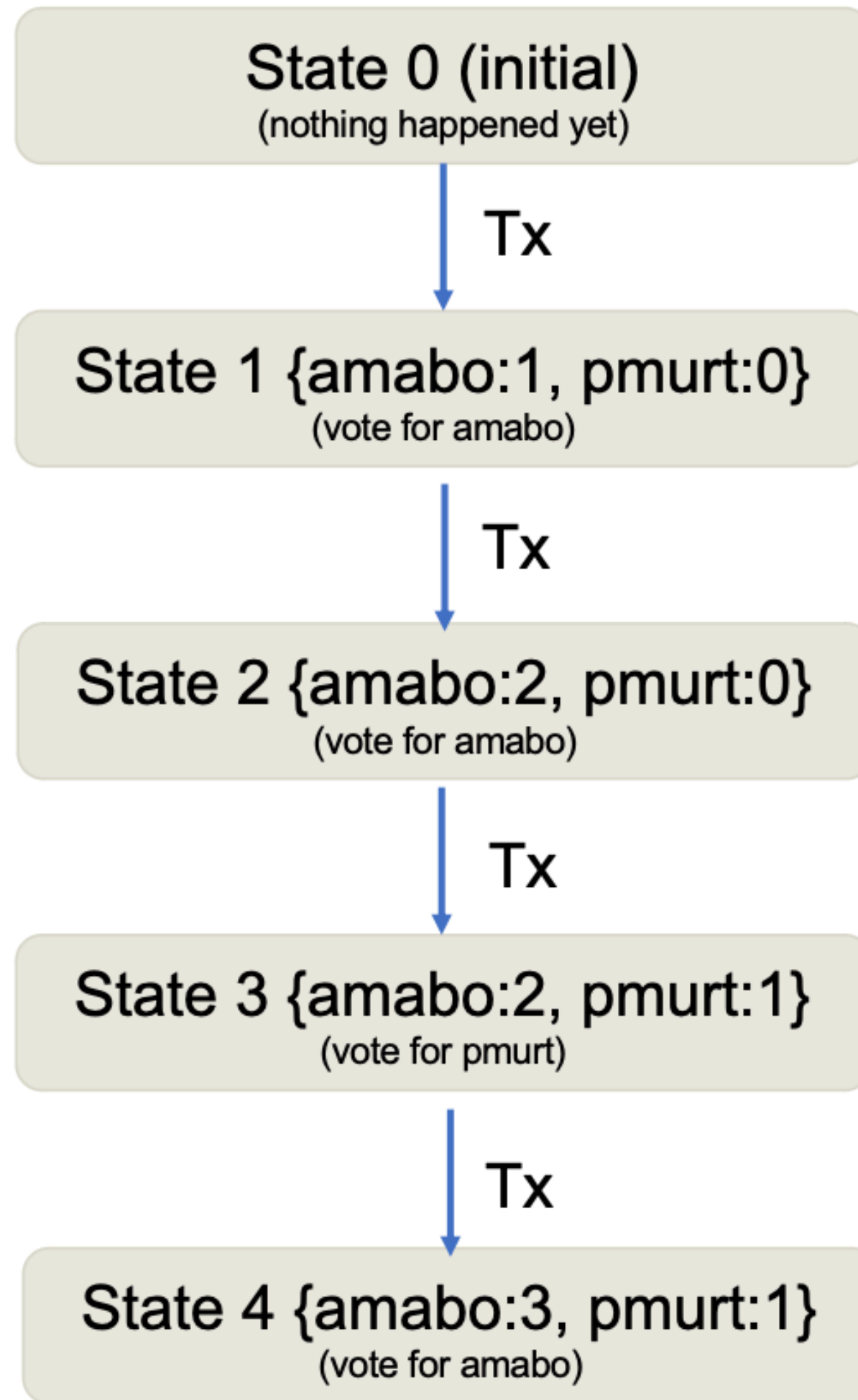


State of the world

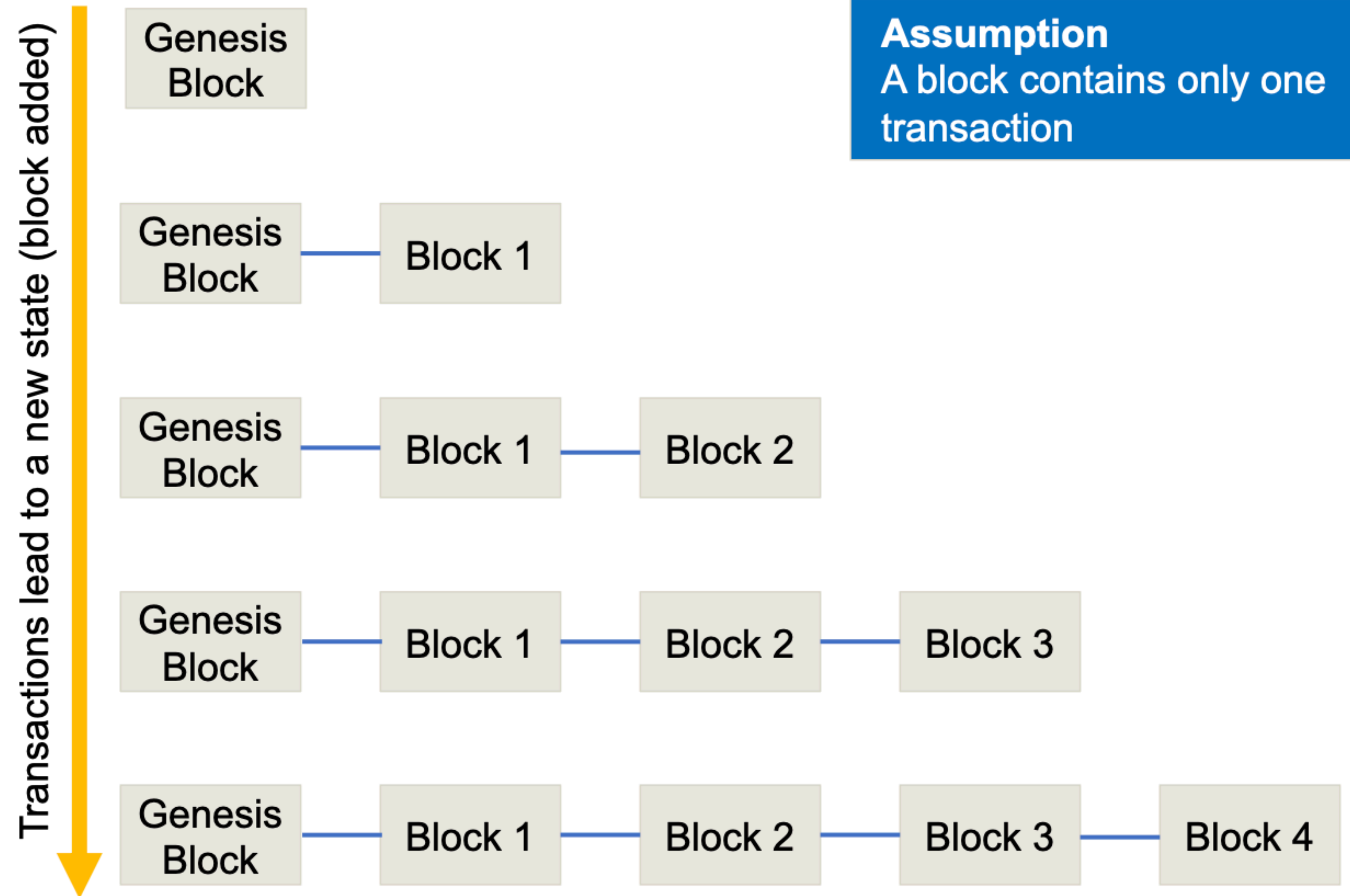


Ethereum: State Machine

State of the world



Blockchain



Assumption
A block contains only one transaction

Ethereum: Accounts

Ethereum has an **account-based ledger** where each **distinct address** represents a separate, unique account.

→ Remember that Bitcoin uses a *transaction-based ledger* model!

Ethereum supports two types of accounts:

Externally Owned Accounts (EOA): *controlled by private keys*

- Accounts that are controlled by a private key do not have any code stored on the blockchain
- **Default wallet of a user¹.**
- They can sign transactions, issue smart contract functions calls, and send Ether.
- The **origin of any transaction** is always an account controlled by a private key.

Smart Contract Accounts: *controlled by their code*

- Smart contracts² are treated as **account** entities with their **own**, unique **address**.
- Contracts³ **can send messages** to other accounts, both externally controlled and smart contracts.
- They **can't issue a transaction** themselves.
- They have a **persistent internal storage** for reading and writing.

Ethereum: Messages vs. Transactions

A **message** is very similar to a transaction. Messages are only sent by contracts and **exist only virtually**.

→ Messages are not mined into a block like transactions!

A message contains:

- The sender of the message (implicit)
- The recipient of the message
- The amount of ether to transfer alongside the message
- An optional data field
- A *GASLIMIT* value

Whenever a **contract calls** a method on **another contract**, a virtual **message** is sent.

Whenever an **EOA calls** a method on a contract, a **transaction** is sent.



Ethereum: Smart Contracts

- A **smart contract** is a **set of functions** that can be called by other users or contracts.
- They can be used to **execute functions, send ether or store data**.
- Each smart contract is an account holding object, i.e., **has its own address**.
- Smart contracts have some peculiarities compared to traditional software.

Security

The **development process** of smart contracts **requires special attention on security**.

Once **deployed**, a **contract** is **publicly accessible** by anyone on the network with the following information:

- Address of the smart contract
 - OPCODE
 - Number of public functions and their hash signature
-
- Furthermore, the whole transaction history is accessible (function calls + actual arguments).
 - **Smart contracts** – once **deployed** – **cannot be changed or patched** anymore.

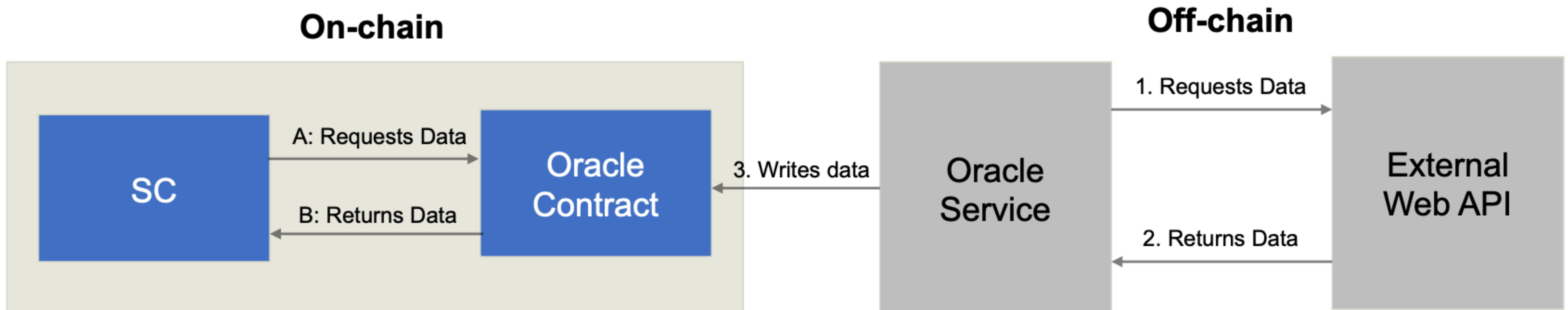
➔ **All contracts deployed on the Ethereum blockchain are publicly accessible and cannot be patched.**

Ethereum: Smart Contracts

Smart contracts can't access any **data** from **outside** the **blockchain** on their own. There are no HTTP or similar network methods implemented to call external services. This is on purpose to **prevent non-deterministic behavior** once a function is called.

→ No function to generate random values!

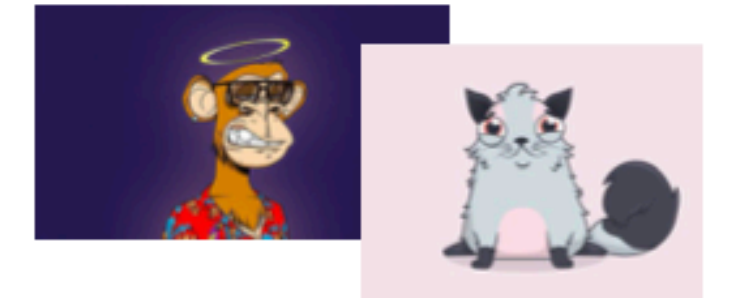
Currently, the **only way** to write **external data** (e.g., weather data, traffic data etc.) to smart contracts is to **use oracles**. Oracles are basically third-party services that verify data from web services and write the data via a specially designed smart contract to the blockchain. Other smart contracts can call the oracle contract to get the data.



Ethereum: Smart Contracts

Fungible and Non-Fungible Tokens (NFT)

- Fungible token systems are currently the largest use case for smart contracts, functioning as a sub-currency of Ethereum and represent a certain asset such as a stock.
 - **Examples:** USDT, USDC, WETH
- NFTs are unique cryptographic tokens that are stored on a blockchain, representing digital assets such as drawings, music, media, virtual fashion items, etc.
 - **Examples:** Bored Ape, CryptoKitties



Decentralized Finance (DeFi)

- Public, permissionless, and interoperable finance ecosystem built on smart contract-enabled blockchains.
 - **Examples:** Decentralized Exchanges (DEX), Lending and Borrowing Platforms



Play-to-Earn Games

- Games where users earn game-native tokens which they can use to buy or trade in-game assets.
 - **Examples:** Axie Infinity, Decentraland



Decentralized Autonomous Organizations (DAO) and Governance

- DAOs are non-hierarchical business models that are collectively owned by a group of members.
- Think of it as a generalization of multi-signature wallets where all decisions are taken based on members' votes. Thus, no single member can unilaterally take a decision (e.g., send a transaction).
- DAOs are useful for starting an organization with people you have limited trust (e.g., only communicating over the internet) since you only need to trust the system design and no other single member.
 - **Examples:** MakerDAO, Friends With Benefits (FWB)



Smart Contracts



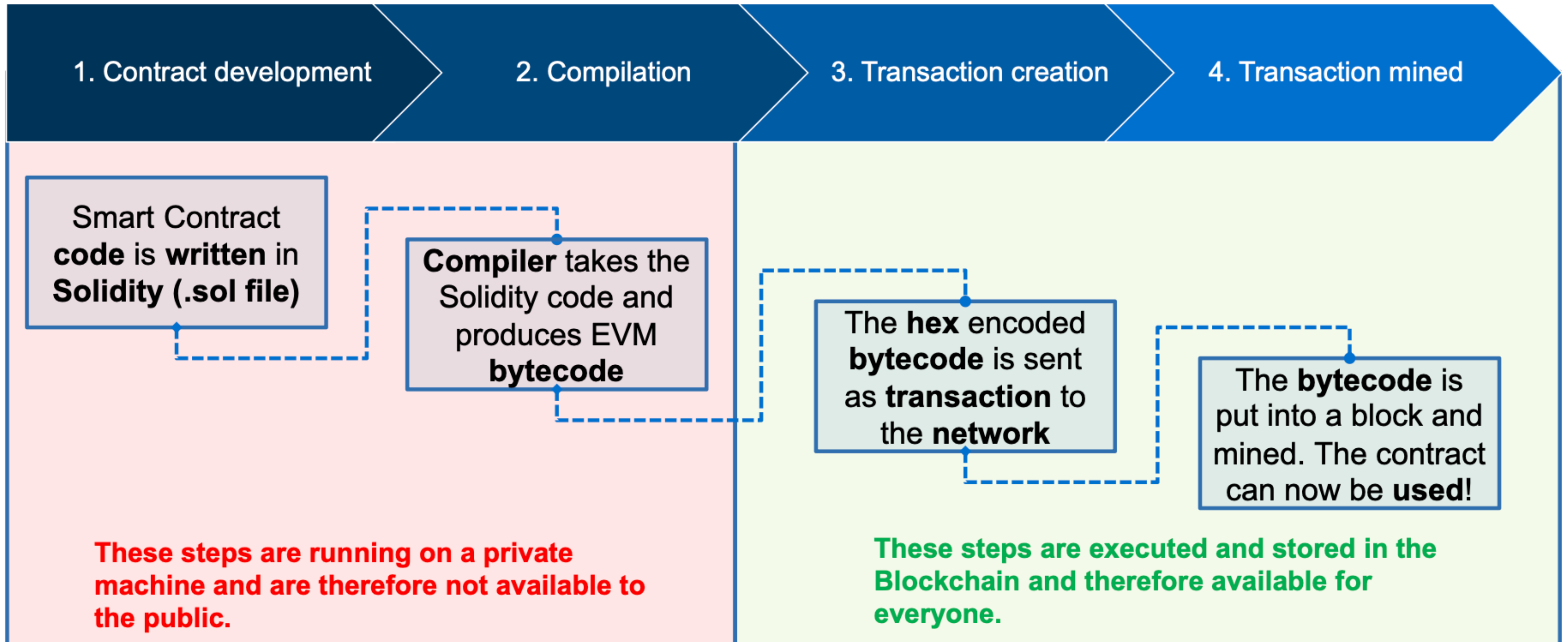
- Solidity is a **high-level language** to write **smart contracts** for **Ethereum**.
- **Contracts**¹ can be defined as **encapsulated units**, similar to classes in traditional **object-oriented** programming languages like Java.
- A **contract** has **its own, persistent state** on the blockchain which is **defined by state variables** in the contract.
- **Functions** are **used to change** the **state** of the contract or to **perform** other **computations**.
- Solidity is **compiled to bytecode** which is **persistent and immutable** once deployed to the blockchain:
 - ➔ **No patch deployment** possible
 - ➔ **Smart contracts** must be **perfected before** using them in **production**!

Smart Contracts

- A smart contract source code is typically **not stored on the blockchain**, only the **bytecode** gets stored.
 - Remember that bytecode represents a series of **EVM opcodes** to be executed.
- Without further analysis, the purpose of a smart contract bytecode is unclear.
- Source code can be made publicly available.
- [Etherscan.io](https://etherscan.io) is the most popular Ethereum block explorer, and a service which verifies source codes and the respective bytecodes.

[illegible]

Smart Contracts



Smart Contracts

File: **BBSE.sol**

```
contract BBSE {
```

```
    struct Tutor {  
        string firstName;  
        string lastName;  
    }  
    mapping (address => Tutor) tutors;  
    address professor;
```

```
    modifier onlyProfessor {  
        require(msg.sender == professor);  
        _;  
    }
```

```
    constructor() public {  
        professor = msg.sender;  
    }
```

```
    function getProfessor() view returns (address) {  
        return professor;  
    }
```

```
    // This function adds a new tutor  
    function addTutor(address tutorAddress,  
        string firstName, string lastName) onlyProfessor {  
        Tutor tutor = tutors[tutorAddress];  
        tutor.firstName = firstName;  
        tutor.lastName = lastName;  
    }
```

```
}
```

State variables

- State variables are permanently stored in the contract's storage.
- Changing the state requires a transaction and therefore costs ether.
- Reading the state of a contract is free and does not require a transaction.

Smart Contracts

File: **BBSE.sol**

```
contract BBSE {  
  
    struct Tutor {  
        string firstName;  
        string lastName;  
    }  
    mapping (address => Tutor) tutors;  
    address professor;  
  
    modifier onlyProfessor {  
        require(msg.sender == professor);  
        _;  
    }  
  
    constructor() public {  
        professor = msg.sender;  
    }  
  
    function getProfessor() view returns (address) {  
        return professor;  
    }  
  
    // This function adds a new tutor  
    function addTutor(address tutorAddress,  
        string firstName, string lastName) onlyProfessor {  
        Tutor tutor = tutors[tutorAddress];  
        tutor.firstName = firstName;  
        tutor.lastName = lastName;  
    }  
}
```

Function modifiers

- Function modifiers are a convenient way to reuse pieces of code.
- Changes the behavior of a function.
- Can execute code either before and/or after the actual function execution.
- The low dash _ indicates where the actual function code is injected.
- Often used for authentication.

Smart Contracts

File: **BBSE.sol**

```
contract BBSE {  
  
    struct Tutor {  
        string firstName;  
        string lastName;  
    }  
    mapping (address => Tutor) tutors;  
    address professor;  
  
    modifier onlyProfessor {  
        require(msg.sender == professor);  
        _;  
    }  
  
    constructor() public {  
        professor = msg.sender;  
    }  
  
    function getProfessor() view returns (address) {  
        return professor;  
    }  
  
    // This function adds a new tutor  
    function addTutor(address tutorAddress,  
        string firstName, string lastName) onlyProfessor {  
        Tutor tutor = tutors[tutorAddress];  
        tutor.firstName = firstName;  
        tutor.lastName = lastName;  
    }  
}
```

Constructor

- The constructor function is executed once when the contract is created through a transaction.
- The function cannot be called after the creation of the contract.
- Usually used to initialize the state of a contract.
- Execution costs gas, and more complex constructors lead to higher deployment costs.

Smart Contracts

File: **BBSE.sol**

```
contract BBSE {  
  
    struct Tutor {  
        string firstName;  
        string lastName;  
    }  
    mapping (address => Tutor) tutors;  
    address professor;  
  
    modifier onlyProfessor {  
        require(msg.sender == professor);  
        _;  
    }  
  
    constructor() public {  
        professor = msg.sender;  
    }  
  
}
```

```
function getProfessor() view returns (address) {  
    return professor;  
}  
  
// This function adds a new tutor  
function addTutor(address tutorAddress,  
    string firstName, string lastName) onlyProfessor {  
    Tutor tutor = tutors[tutorAddress];  
    tutor.firstName = firstName;  
    tutor.lastName = lastName;  
}
```

```
}
```

Functions

- Functions are used to change the state of a contract.
- Can also be used to read the state of the contract.
- Consist of a name, a signature, a visibility, a type, a list of modifiers, and a return type.

Formal definition:

```
function (<parameter types>)  
{internal|external|public|private}  
[pure|constant|view|payable]  
[(modifiers)]  
[returns (<return types>)]
```

Smart Contracts

In Solidity, functions can be declared with four different visibility types.

External

External methods can be called by other contracts and via transactions issued by a certain wallet. Methods declared as external **are always publicly visible** and **can't be called directly by the contract itself**.

Public

Public can **be called internally** by the contract itself but also **externally** by other contracts and via transactions. **State variables** which are defined as public will, **by default, have getter** method created automatically by the compiler.

Internal

Internal methods can only be accessed by the contract itself or by any contract derived from it. They are not callable from other contracts or via transactions.

Private

Private methods can **only** be called **internally** by the contract that owns the method. **Derived contracts cannot access** a private method of their parent contract.

Smart Contracts

Storage	Memory	Stack
▪ The storage is comparable to a hard drive. It keeps data between function calls and is persistent for each smart contract. This way, every execution on the contract has access to the data previously saved in the storage area.	▪ The memory is comparable to a computer's RAM. It is a temporary storage location for data. Once the execution is complete, the memory is cleaned for the next execution.	▪ As EVM is a stack machine rather than a register machine, all computations are done on a data region called the stack. It has a maximum of 1024 elements and comprises 256-bit words.

Variables

- i. **State variables** are saved in **storage** by default.
- ii. **Value-type local variables** and **function arguments** (int, bool, address, ...) are stored in **memory**.
- iii. **Reference-type local variables** (array, struct, string, ...) can be stored in **memory** or **storage**.
- iv. **Reference-type function arguments** can be stored in **memory** or **calldata** (a **read-only data location** similar to memory but only available to function arguments).

Note

- Reading from the storage is **expensive**. Initializing and **modifying** storage is even more expensive. Thus, you should limit the data you keep in persistent storage to what the contract requires.
- Compared to the gas consumption of storage, memory has a relatively **low consumption**. Hence, it is cost-efficient to do intermediate calculations in memory and then save the result to storage if necessary.

Smart Contracts

Storage

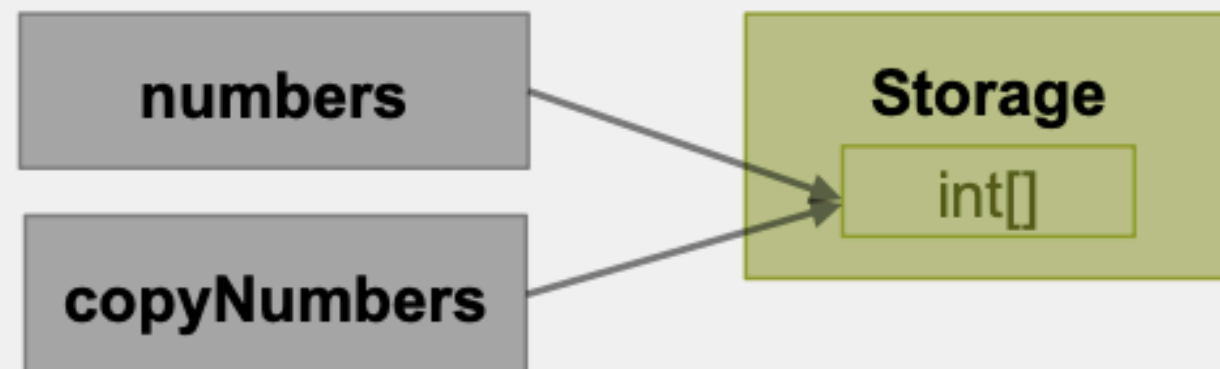
```
1 pragma solidity >=0.7.0 <0.9.0;
2
3 contract ExampleStorage {
4
5     // Initializing an integer array 'numbers'
6     // Stored in 'storage' as it is a state variable
7     int[] public numbers;
8
9     // A function to insert values into the 'numbers' array and modify them
10    function insertAndModify() public {
11        numbers.push(1);
12        numbers.push(2);
13
14        // Creating a pointer to the storage location of 'numbers'
15        int[] storage copyNumbers = numbers;
16
17        // Assign a new value to index 0 of the new instance
18        copyNumbers[0] = 5;
19    }
20 }
```

Output

numbers	0
0: int256:	5

numbers	1
0: int256:	2

- *copyNumbers* is a **pointer** to the *numbers* array's **storage location** (like a **shallow copy**)



Memory

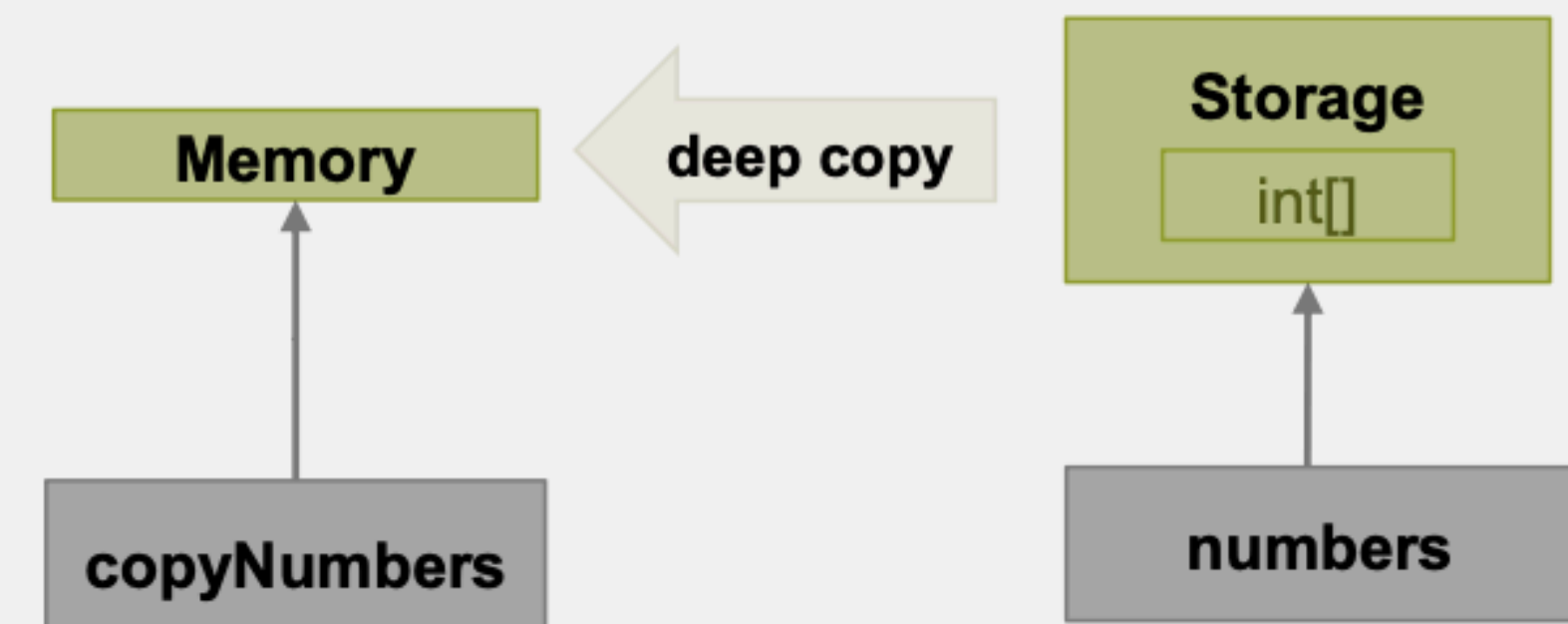
```
1 pragma solidity >=0.7.0 <0.9.0;
2
3 contract ExampleMemory {
4
5     // Initializing an integer array 'numbers'
6     // Stored in 'storage' as it is a state variable
7     int[] public numbers;
8
9     // A function to insert values into the 'numbers' array and modify them
10    function insertAndModify() public {
11        numbers.push(1);
12        numbers.push(2);
13
14        // Creating a local copy of the 'numbers' array
15        int[] memory copyNumbers = numbers;
16
17        // Assign a new value to index 0 of the new instance
18        copyNumbers[0] = 5;
19    }
20 }
```

Output

numbers	0
0: int256:	1

numbers	1
0: int256:	2

- The keyword **memory** must be used in order to make sure that *copyNumbers* **only exists locally**



Smart Contracts

Solidity provides special function type declarations besides the default one.

View function

Functions that are declared as view are read-only, i.e., they don't modify any state variable nor alter the state of the blockchain. However, they can read from the state variables.

```
uint state = 5;  
function add(uint a, uint b) public view returns (uint sum) { return a + b + state }
```

Pure function

Pure functions can be seen as a subset of view functions that don't modify the state but also don't read from the state.

```
function add(uint a, uint b) public pure returns (uint sum) { return a + b }
```

Fallback function

A contract can have one unnamed fallback function. The fallback function is called when no other function matches the function call (e.g., when Ether is sent to a contract which doesn't have a *receive*¹ function). A special feature of this function is that it can't have any parameters and doesn't return anything.

```
function() { /* ... */ }
```

Payable function

By default, it is not possible to send Ether to a function, and the function will revert the transaction. This behaviour is intentional to prevent Ether that is accidentally sent from getting lost. However, sometimes it is necessary to pay a contract, e.g., in the case of an ICO. Therefore, Solidity implements *payable* functions.

Smart Contracts

Solidity supports the inheritance of contracts. Technically, the solidity compiler copies the code from the parent contract **to the subcontract** and creates a single piece of bytecode which is deployed on the blockchain.

Solidity also **supports multiple inheritances** for a contract. In this case, the compiler just copies all parent contracts together and creates a single contract that is compiled to bytecode and deployed to the blockchain. Once a contract is deployed, it is not possible to detect from the bytecode whether a contract made use of inheritance or not.

If a parent contract contains a function that is also present in the subcontract, then the functions are overloaded. In case both functions have the same signature, the subcontract function will override the parent's function. However, the parent function can still be explicitly accessed using the super keyword.

Smart Contracts

Solidity uses, similar to Python, the **C3 superclass linearization** algorithm to define the order of the inherited functions. There is no implicit order for parent classes, the order is defined by the developer.

Assume the following inheritance graph

